

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Tesi di laurea di I livello in
Informatica

Human-Computer Interaction & Music: A Machine Learning approach for music performance

Relatore:

Rocco Zaccagnino

Candidato:

Daniele Perrella

Mat. 05121 05206

ANNO ACCADEMICO 2019/2020

INDICE

1	INTRODUZIONE	4
2	RELATED WORKS	7
3	BACKGROUND	12
3.1	Natural User Interfaces	12
3.2	Il Motore Grafico Unity 3D	14
3.2.1	Assets	17
3.2.2	GameObjects & Components	18
3.2.3	Scene	19
3.2.4	Scripting: La classe MonoBehaviour	19
3.2.5	Audio in Unity	22
3.3	Gesture Recognition	23
3.4	Leap Motion	25
3.5	Artificial Neural Networks	26
3.5.1	Funzionamento	26
3.5.2	Ottimizzazione	27
3.5.3	Struttura di una rete neurale	27
3.6	Paradigmi di apprendimento	28
3.7	Fondamenti di Acustica	30

3.8	Digital Audio	31
4	MARCOSMILES 2.0	33
5	IL NOSTRO APPROCCIO	35
5.1	Costruzione del dataset	35
5.2	Compatibilità delle librerie	37
5.3	Classificatore Machine Learning	39
5.3.1	Albero decisionale	40
5.3.2	Random Forest	41
5.4	Feature Scaling	42
5.5	Strategie Utilizzate	45
5.5.1	Formazione del dataset	45
5.5.2	Paradigma di Apprendimento	47
5.5.3	Script di Machine Learning	47
5.5.4	Metodo di scaling	53
5.5.5	Funzione di attivazione	53
5.5.6	Feature Recognition	53
6	RISULTATI E DISCUSSIONI	61
6.1	Analisi Overfitting e Differenze tra dati non scalati e dati scalati	61
6.1.1	1 Ottava con dati non scalati	62
6.1.2	1 Ottava con dati scalati	64
6.1.3	2 Ottave con dati non scalati	66
6.1.4	2 Ottave con dati scalati	68
6.2	Differenze di tempo nella fase di learning	70
6.3	CONCLUSIONI	72
6.4	Lavori futuri	73

CAPITOLO 1

INTRODUZIONE

Viviamo in un'epoca in cui la tecnologia ha completamente rivoluzionato molti degli aspetti della vita giornaliera di un individuo. In un contesto simile, fornire accesso agli strumenti informatici di uso comune ad ogni tipo di utente, è di fondamentale importanza. Requisiti come l'usabilità, intesa come la facilità di interazione tra l'uomo e uno strumento, e accessibilità, ovvero la caratteristica di un servizio di essere fruibile da tutte le categorie di utenti, rappresentano dei punti cardine in fase di progettazione di un sistema informatico, e sono oggetto di studio della *Human Computer Interaction* (HCI). Il costante sviluppo in questi ambiti ha portato alla nascita di strumenti che possano permettere l'utilizzo dei dispositivi elettronici a chiunque, scavalcando barriere derivanti da problemi quali il gap inter-generazionale, difficoltà d'utilizzo, mancanza di accessibilità e scarsa attrattività. In particolare negli ultimi anni sono stati esplorati nuovi modi di interagire con i computer, progettando nuove interfacce basate sull'utilizzo di ambienti 3D, gestures e contextual awareness, al fine di rendere l'interazione più semplice, intuitiva e accattivante. In tale contesto, sono state oggetto di numerosi studi negli scorsi anni le *Natural User Interfaces* (NUI), descritte alla presentazione della conferenza "Predicting the Past" del 2008 come "la

prossima fase evolutiva dopo il passaggio dall’interfaccia a riga di comando (CLI) all’interfaccia grafica utente (GUI)” [1]. L’obiettivo principale delle NUI, è quello di superare le classiche interazioni fisiche legate all’utilizzo di mouse e tastiera, per sostituirle con un più ampio insieme di modalità di comunicazione innovative, che prevedano l’utilizzo di azioni, gesti e movimenti naturali da parte dell’utente, per controllare la macchina che sta utilizzando [2]. La popolarità di queste interfacce innovative è in continua crescita, soprattutto grazie alla diffusione di dispositivi low-cost per il tracking dei movimenti, come il Leap Motion o il Kinect, e dispositivi che supportano il riconoscimento della voce come Amazon Alexa. Nonostante la già grande popolarità nel mercato dei videogiochi, queste tecnologie sono state impiegate in vari settori quali la riabilitazione, la robotica, e l’home automation.

L’obiettivo di questa tesi è quello di utilizzare questo tipo di interfacce, insieme ad un dispositivo per il riconoscimento dei movimenti delle mani, nell’ambito della *Computer Music*. La computer music o musica informatica è un ramo della musica elettroacustica che impiega sistematicamente la programmazione su elaboratore numerico. In senso più esteso, qualunque produzione musicale che utilizzi il mezzo informatico può definirsi computer music [3]. Pietro Grossi, pioniere italiano dell’ingegneria, nel 1967, alle prime esperienze ufficiali di computer music presso la Olivetti General Electric a Pregnana Milanese dichiarò: “...Ho preso il quinto Capriccio di Paganini, ho perforato le schede e, fatto il pacco di schede, consegnate al computer, il computer le suonava. Il computer ha suonato subito alla perfezione il testo che gli avevo dato, e immediatamente dopo, ho potuto fare di quei suoni quello che volevo. Questo per me era un salto incredibile. È stata un’emozione straordinaria...” [4]. Quest’ambito è in continua evoluzione, e negli ultimi anni diversi musicisti hanno mostrato interesse nella possibilità di utilizzare dei controller che siano in grado di personalizzare il modo in cui ci si esprime “musicalmente”. Un esempio degno di nota è la performance eseguita dalla nota cantante americana Ariana Grande nel 2015 [6], in cui fa uso dei guanti Mi.Mu Gloves [7] per armonizzare la melodia che sta cantando,

mettendo in luce le grandi potenzialità derivanti dall'utilizzo di un controller per l'espressività.

Per la maggior parte degli strumenti musicali reali, è possibile rappresentare le note musicali in termini di configurazioni della mani. Ogni configurazione contiene informazioni circa la posizione e la direzione di mano e dita, ad un certo istante di tempo. Una modalità semplice e naturale di associare gesti dell'utente alla riproduzione di musica real-time è proprio attraverso l'associazione di configurazioni delle mani a delle specifiche note musicali. Su questo principio è basato il sistema MarcoSmiles [5], uno strumento che consente di poter suonare uno strumento virtuale attraverso dei semplici gesti delle mani, senza l'uso di uno strumento musicale reale, e in maniera del tutto personalizzabile, rivolto principalmente a persone affette da disabilità motorie. Per il funzionamento del sistema è necessario un LeapMotion Controller che avrà il ruolo di acquisire informazioni circa la posizione della mani dell'utente, per poi fornirle ad una Rete Neurale Artificiale (ANN) opportunamente addestrata che provvederà a realizzare l'associazione posizione - nota musicale real-time. Questo lavoro di tesi ha come obiettivo quello di superare i limiti di tale progetto sulla parte della rete neurale, definendo ed implementando un classificatore che permetta un riconoscimento delle gestures più accurato ed efficace. La scelta della libreria Scikit-Learn, come sarà spiegato nei capitoli successivi, è stata una scelta abbastanza forzata, ma che si è rivelata eccellente. Nel Capitolo 2 discuteremo di alcuni lavori rilevanti in questo settore, tracciando un percorso storico sull'evoluzione degli strumenti musicali non convenzionali. Nel Capitolo 3 verranno descritti in principali ambiti che verranno trattati in questo lavoro. Nel quarto capitolo verrà fornita una descrizione del sistema e degli obiettivi che si pone. Il quinto capitolo è dedicato allo sviluppo del modulo oggetto di questa tesi. Infine, nel sesto capitolo verranno discussi i risultati ottenuti, e le possibili prospettive sui lavori futuri per estendere il sistema.

CAPITOLO 2

RELATED WORKS

Negli ultimi anni sono stati effettuati molti studi che hanno evidenziato una vasta scelta di possibilità nel produrre musica in maniera non convenzionale, attraverso strumenti elettronici sia fisici che virtuali. Tuttavia per comprendere al meglio la storia e l'evoluzione degli strumenti musicali accessibili è necessario fare un passo indietro; quando l'utilizzo dell'elettricità era ancora nelle prime fasi di sperimentazione, e la lampadina doveva ancora essere inventata, alcune persone erano già a lavoro per utilizzare l'elettricità nel processo di creazione del suono. A causa della mancanza delle tecnologie di registrazione, questi primi esperimenti furono progettati esclusivamente per performance live [8]. Nel 1919 è nato Theremin [9], il primo strumento che non richiede contatto fisico con l'esecutore. Il Theremin è stato inventato dal fisico sovietico Lev Sergeevič Termen nel 1919, e si basa sull'utilizzo di oscillatori elettronici che lavorano in isofrequenza al di fuori dello spettro udibile. Il musicista provoca un alterazione degli oscillatori inserendo le sue mani nel campo d'onda, producendo dei suoni sul principio fisico del battimento, questa volta nel campo delle frequenze udibili. Lo strumento è sostanzialmente composto da 2 antenne annesse ad un contenitore in cui alloggia l'elettronica dello strumento. Le antenne controllano rispettivamente

2. RELATED WORKS

frequenza e intensità del suono prodotto. Il Theremin ha rappresentato una vera e propria rivoluzione nell'ambito della musica, ed è stato utilizzato negli anni successivi in diversi contesti e in numerosi generi musicali.

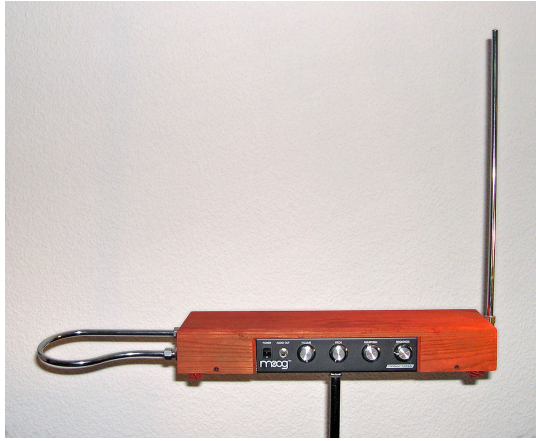


Figura 2.1: Theremin [9]

Verso la prima metà degli anni 70 è stato sviluppato l'iconico Mellotron [10], uno strumento a tastiera pensato per riprodurre strumenti musicali tipici delle orchestre sinfoniche, e voci umane. Questo strumento può essere considerato come il padre dei campionatori elettronici, comparsi all'inizio degli anni 80. Il mellotron genera i suoni tramite nastri registrati. Quando viene premuto un tasto, il nastro collegato viene spinto sulla testina di riproduzione, come in un registratore a nastro. Finché il tasto resta abbassato, il nastro scorre sulla testina e il suono viene riprodotto. Quando il tasto viene rilasciato, una molla fa tornare indietro il nastro alla sua posizione di riposo. In seguito ai progressi dell'industria elettronica, furono prodotti numerosi strumenti elettronici di vario genere, tra i quali possiamo ricordare il Suzuki Omnichord [11]. Il suo funzionamento è semplice: la mano sinistra preme i tasti che formano gli accordi, mentre la destra sfiora il touchpad che sostituisce le corde. L'omnichord offre anche altre possibilità per ottenere suoni, anche utilizzando soltanto la tastiera. È uno dei primi strumenti elettronici ad essere considerato accessibile, e viene utilizzato tutt'ora in musicoterapia. Attualmente nell'ambito della Computer Music rivestono un ruolo di grande importanza sensori di varie tipologie. Tra i più utilizzati troviamo i sensori

2. RELATED WORKS

tattili, fotocamere e accelerometri. Un importante esempio è rappresentato dal guanto KAIKU [13], un sensore capacitivo indossabile, che risponde alla prossimità e al contatto di conduttori, come il corpo umano.



Figura 2.2: Kaiku Glove durante una performance musicale [13]

Il sensore viene principalmente utilizzato come controller MIDI, quindi può essere utilizzato in vari contesti, e dispone di un software dedicato che permette di emulare diversi tipi di strumenti. Un tipico esempio di utilizzo consiste nel mappare delle note a delle zone del guanto, che possono essere sfiorate con le dita o altri oggetti conduttori per inviare un segnale MIDI al dispositivo a cui è connesso il guanto, in modo che venga suonata la nota desiderata. Sono presenti anche altri tipi di sensori indossabili, e vengono spesso utilizzati per la riabilitazione e la musicoterapia. Un altro strumento usato in questi due ambiti è il software AUMI [12]. Quest'ultimo permette all'utente di poter riprodurre suoni e frasi musicali attraverso movimenti e gestures, sfruttando la fotocamera del dispositivo sul quale è in uso. Il software prevede un approccio personalizzabile attraverso l'utilizzo di effetti e la possibilità di variare i suoni utilizzati. Il sistema è progettato principalmente per essere utilizzato da bambini che hanno disabilità fisiche. Molto diffuso è anche il Kinect [14], un sensore sviluppato da Microsoft per le console Xbox. Il Kinect si avvale di una telecamera RGB e di una telecamera a radiazione infrarossa con un risoluzione di 480p ciascuna, ed una capacità di lettura del movimento a 30fps (frames per second). Per il calcolo della lontananza degli oggetti, invece, Kinect utilizza di uno scanner 3D, in combinazione con gli infrarossi, che però è dotato di una risoluzione di 320×240 pixel. Kinect dispone inoltre di quattro microfoni utilizzati dal

2. RELATED WORKS

sistema per l'elaborazione dati dell'ambiente in cui ci si trova, che permettono la cancellazione dell'eco e riduzione del rumore. Il Kinect è stato utilizzato in numerosi progetti di vario genere nell'ambito della Computer Music, spesso facendo uso dell'Hand Gesture Recognition. Un esempio consiste nel lavoro effettuato dalla National Taiwan University of Science and Technology [15], ovvero un sistema capace di simulare la conduzione musicale in maniera real-time, sfruttando un classificatore. Il sensore utilizzato nel nostro sistema è il LeapMotion Controller. L'applicazione del sensore LeapMotion nell'ambito della Computer-Music è stata oggetto di diversi studi. In primo luogo è stata esplorata la possibilità di utilizzare il LeapMotion assieme al protocollo MIDI, sfruttando la Gesture-Recognition al fine di inviare segnare segnali per modulare determinati parametri all'interno di una DAW o di VST. Un esempio di questo tipo di applicazione è lo strumento AeroMIDI [16], che permette di creare un'interfaccia completamente personalizzabile, in cui è possibile mappare parametri di un Virtual Instrument o di qualsiasi altra funzione, a gesti delle mani, con la possibilità di controllarne fino a 6 per mano. Il sistema AeroMIDI fa parte della categoria dei numerosi strumenti che utilizzano il LeapMotion come controller MIDI, e garantiscono all'utente una maggiore espressività durante la performance musicale. Uno studio sull'Gesture Recognition utilizzando il LeapMotion è stato effettuato dalla Dalian University of Technology [17], e ha evidenziato la possibilità di sviluppare un classificatore in grado di riconoscere accuratamente gesture scelte dall'utente in fase di training. Un altro progetto che utilizza il LeapMotion nell'ambito della Computer - Music è Virtual Piano [18]. Si tratta di un software basato sulla gesture recognition, che permette di poter suonare un pianoforte virtuale attraverso delle gesture delle mani predefinite, e dispone di alcune semplici canzoni da poter suonare.

2. RELATED WORKS

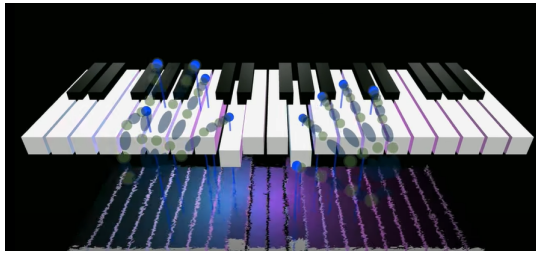


Figura 2.3: Virtual Piano [18]

E' importante sottolineare che la principale differenza fra i sistemi sopracitati rispetto a MarcoSmiles è che la configurazione delle mani associata ad una determinata nota è scelta dall'utente in base alle sue preferenze. L'utente dovrà allenare il sistema ad imparare i gesti che intende utilizzare, per poi successivamente utilizzare per suonare in modo naturale, come se fosse uno strumento reale.

CAPITOLO 3

BACKGROUND

3.1 Natural User Interfaces

Come già accennato nel paragrafo introduttivo, le Natural User Interfaces sono interfacce invisibili all'utente, caratterizzate da interazioni semplici e di facile apprendimento. La parola "Natural" si riferisce proprio all'obiettivo delle NUI, cioè rendere l'interazione con l'utente del tutto naturale, attraverso l'utilizzo di capacità umane quali tatto, vista, utilizzo della voce, movimento, fino ad arrivare a funzioni cognitive più complesse come percezione ed espressione. Un altro aspetto importante riguardo le NUI consiste nella selezione di gestures che l'utente conosce e utilizza durante la vita di tutti i giorni. Ciò avviene in modo da ridurre il carico cognitivo dell'utilizzatore, cioè lo sforzo richiesto nel pensare o effettuare un ragionamento prima di usare una determinata gesture all'interno dell'interfaccia, in maniera tale da eliminare qualsiasi tipo di distrazione. Da tenere in grande considerazione è anche il contesto in cui la NUI viene utilizzata. Durante la fase di progettazione bisogna tener conto dell'impossibilità nel creare un' interfaccia che risulti naturale in tutti i contesti e per tutti gli utenti. Di conseguenza la scelta delle gestures deve essere effettuata alla luce del contesto in cui l'interfaccia viene utilizzata,

e tenendo conto dello skill level dell'utilizzatore medio. Le NUI presentano diverse caratteristiche [19]:

- **User-centered:**

devono adattarsi alle necessità dell'utente. In un tradizionale approccio uomo-macchina, le persone vengono considerate come operatori che si adattano alla macchina; in un approccio moderno le persone vengono considerate come utenti che dialogano con la macchina, tuttavia senza averne un controllo effettivo; in un approccio che utilizza le NUI le persone sono partecipanti attivi, e la macchina risponde alle varie interazioni dell'uomo.

- **Multi-channel:**

Le interfacce Multi-channel fanno uso di uno più sensori per determinare le intenzioni dell'utente durante l'interazione, al fine di migliorarla e renderla più naturale. L'utente può comunicare attraverso diverse modalità quali vista, tatto, udito, olfatto, ed equilibrio. I "mezzi" di cui si serve sono mani, bocca, occhi, piedi etc. Nelle classiche interazioni uomo macchina, sono coinvolti esclusivamente vista e tatto, attraverso gli occhi alle mani. Per rendere l'interazione più efficiente e al fine di non affaticare i "mezzi" dell'utente, quest'ultimo dovrebbe poter scegliere il miglior mezzo per comunicare con la macchina.

- **Imprecisione:**

Una tecnologia che offre un'interazione precisa riesce a comprendere l'intenzione dell'utente nella sua interezza. Spesso le azioni o i pensieri delle persone non risultano molto precisi, dunque la macchina deve essere in grado di capire le richieste dell'utente, tenendo conto anche del possibile margine d'errore.

- **Voice-based interaction:**

Il linguaggio umano rappresenta il modo più naturale ed efficiente per condividere informazioni. In media durante la giornata, circa 75% della

comunicazione effettuata un individuo avviene attraverso la voce. Inoltre il riconoscimento di segnali audio è significativamente più veloce del riconoscimento di segnali visivi.

Si può dunque dire che la voce rappresenta il più importante mezzo di interazione tra una macchina e un essere umano.

- **Behavior-based Interaction:**

Nel processo di interazione, oltre alla voce, le persone utilizzano anche i movimenti del corpo per esprimersi. L'interazione uomo-macchina basata sul comportamento consiste nella capacità di una macchina di riconoscere il comportamento umano attraverso posizionamento, movimento ed caratteristiche espressive del corpo umano, e nel rispondere in maniera "intelligente" a variazioni di esse. Con questo tipo di interazione il comportamento dell'utente può essere predetto dalla macchina.

Il noto fondatore di Microsoft, Bill Gates, ha dichiarato in merito alle NUI: "Fino ad ora ci siamo sempre dovuti adattare ai limiti imposti della tecnologia e adattare il modo in cui lavoriamo con i computer ad un insieme di procedure e convenzioni. Attraverso le NUI, per la prima volta saranno i computer ad adattarsi ai nostri bisogni e preferenze, e per la prima volta gli essere umani interagiranno con le macchine in maniera del tutto naturale" [2].

3.2 Il Motore Grafico Unity 3D

Il nostro sistema è basato su Unity3D, un motore grafico multi-piattaforma che consente lo sviluppo di videogiochi e altri contenuti interattivi, quali visualizzazioni architettoniche o animazioni 3D in tempo reale. Unity permette all'utente di creare ambienti sia in 2D che in 3D. Uno screening del 2018 rileva che Unity è stato utilizzato per la creazione di circa la metà dei giochi mobile disponibili sul mercato e del 60% dei giochi in realtà aumentata e VR. È inoltre stato adottato da altri ambiti industriali che non

3. BACKGROUND

rientrano in quello del settore dei videogiochi, come cinemagrafia, architettura, ingegneria e costruzione. Un vantaggio non di poco conto è che Unity mette a disposizione versioni gratuite per studenti e per privati con entrate inferiori ai 100.000\$ annuali.

L'Engine Unity3D, rende agevole la disposizione di livelli, la creazione di menu, l'esecuzione di animazioni e l'organizzazione dei progetti. Inoltre, integra al proprio interno la possibilità di scrivere ed eseguire scripts, scritti in C#.

L'interfaccia grafica è molto ben organizzata e i pannelli di lavoro sono totalmente personalizzabili. È proprio attraverso la selezione ed il trascinamento dei pannelli che si attiva tale personalizzazione.

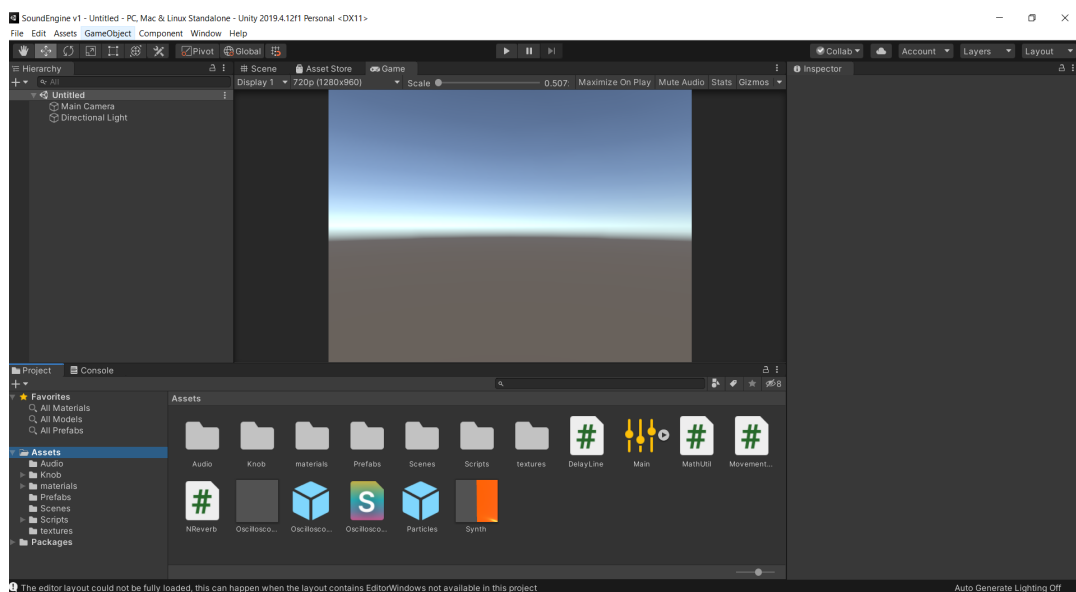


Figura 3.1: Scena vuota Unity3D

Questi i principali pannelli all'interno di Unity:

- Il pannello “Project” è quello in cui tutti gli assets del progetto vengono immagazzinati. Da qui è possibile organizzare i file che usiamo all'interno di un progetto.

Ogni volta che importiamo degli assets appariranno qui.

3. BACKGROUND

- Il pannello “Hierarchy” è quello in cui gli assets vengono organizzati all’interno di una scena. Ogni Asset, può essere trasportato dal pannello “Project” a quello “Hierarchy”, in modo tale da inserire assets all’interno di una scena. Gli elementi all’interno di questo pannello sono chiamati GameObjects.
- Il pannello “Inspector” permette di ispezionare e regolare tutti gli attributi di un GameObject. Da questo pannello si possono regolare per un GameObject, ad esempio, la posizione, la rotazione, le proprietà di gravità all’interno del gioco, e tutte le altre caratteristiche. Questo è fatto attraverso l’ausilio di Components, ossia, componenti che ci permettono di specificare determinate proprietà per il GameObject selezionato. Anche gli scripts vengono trattati come se fossero Components e, dunque, aggiunti tramite questo pannello.
- Il pannello “Scene” permette di osservare e di arrangiare gli assets all’interno del gioco e dello spazio 3D. Da qui, è possibile spostarci all’interno dello spazio 3D utilizzando il mouse.
- Il pannello “Console” è letteralmente la console di Unity. Molto utile in fase di debugging degli scripts.

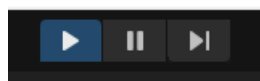


Figura 3.2: Pulsanti Play

Per far partire il proprio gioco si preme sul pulsante Play posto nella parte alta della schermata. Si può mettere in pausa il gioco (usando il pulsante Pause) in modo tale da ispezionare la scena. Inoltre è possibile passare allo step successivo del gioco, utilizzando il pulsante Step.

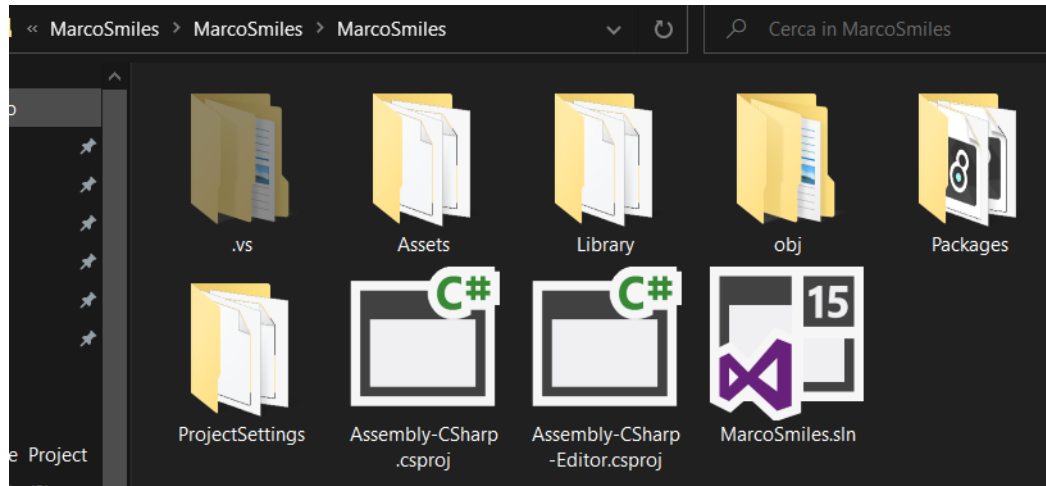


Figura 3.3: La directory di un progetto Unity

In Unity, un progetto è una cartella ordinaria, contenente ogni risorsa utilizzata dal gioco. Infatti, come risultato della creazione di un progetto nuovo, avremo una cartella che contiene delle sottocartelle chiamate rispettivamente “Assets”, “Library” e “ProjectSettings”.

3.2.1 Assets

Gli Assets sono una qualsiasi risorsa utilizzata dal gioco. Questo include modelli 3D, Materials, Textures, Audio, Scripts, Fonts etc..

Unity gestisce in modo robusto ed intelligente l’importazione di Assets in un progetto. Gestisce al meglio la maggior parte dei formati di file, risolvendo uno dei difetti che tradizionalmente ha caratterizzato vecchi game engine, ossia quello di essere vincolati all’estensione del file quando si cerca di importarlo all’interno di un progetto.

Unity ha funzionalità built-in che permettono di creare semplici oggetti come Sfere o Cubi, ma quando c’è la necessità di lavorare con oggetti più complessi, l’ideale è importare modelli 3D all’interno del proprio progetto. L’Engine riesce a gestire la maggior parte dei formati 3D, inclusi Maya, 3D Studio Max, Blender e FilmBox, mantenendo tutti i materiali e le textures intatte. Stesso discorso è valido per ciò che riguarda formati di immagini, supportando PNG, JPEG, TIFF e addirittura file PSD con livelli Photoshop, e per i formati di

file audio, supportando formati quali WAV, AIFF, MP3 e OGG.

Una lista completa di tutti i formati che possono essere importati all'interno di Unity la si può trovare al seguente link:

<http://unity3d.com/unity/editor/importing>

È opportuno notare che Unity mette a disposizione un Assets Store, dove si possono acquistare o scaricare gratuitamente Assets e risorse per i propri progetti.

3.2.2 GameObjects & Components

Il concetto di **GameObject** è probabilmente il concetto più importante all'interno dell'editor di Unity. Ogni oggetto all'interno del gioco è un **GameObject**. Nonostante ciò, un **GameObject** non può fare nulla all'interno del gioco se non gli si conferiscono proprietà. Possiamo descrivere un **GameObject** come una scatola che permette di contenere funzionalità. Queste funzionalità sono descritte dai **Components**. A seconda di che tipo di oggetto si vuole creare, si aggiungono diverse combinazioni di **Components** all'interno di un **GameObject**.

Un **GameObject** contiene sempre una Component di tipo Transform che non è possibile rimuovere. Questa rappresenta la posizione e l'orientamento dell'oggetto.

Altre Components che danno funzionalità agli oggetti possono essere aggiunte dal menu rinominato "Component" o da uno script.

È importante notare che esistono degli oggetti pre-costruiti chiamati Primitive Objects. Questi si trovano nel menu GameObject/3D Object e permettono di creare degli oggetti semplici come una camera, una sfera o un cilindro.

Per default, ogni scena viene inizializzata con un GameObject chiamato Main Camera. Questo oggetto è configurato per comportarsi come camera principale all'interno del gioco. Contiene una componente di tipo Transform, una di tipo Camera e un'altra di tipo AudioListener che permette di percepire i suoni che si trovano nell'ambiente 3D di una scena.

3.2.3 Scene

Le scene rappresentano determinati ambienti del gioco. Sono i “luoghi” che contengono gli Assets e descrivono la schermata del gioco. Mentre il pannello “Scene” è ideale per arrangiare gli Assets all’interno dello spazio 3D, il pannello “Hierarchy” è adatto ad organizzare i GameObjects all’interno della scena, descrivendo i contenuti della scena corrente tramite relazioni padre-figlio fra i vari GameObjects.

3.2.4 Scripting: La classe MonoBehaviour

Gli scripts sono anche chiamati “behaviours” in Unity. Permettono di rendere gli assets ed i GameObjects interattivi all’interno di una scena. Più script possono essere collegati ad uno stesso GameObject, permettendo un facile riuso di codice.

Unity supporta lo scripting in C#.

La classe **MonoBehaviour** è la classe base da cui deriva ogni script in Unity.

3. BACKGROUND

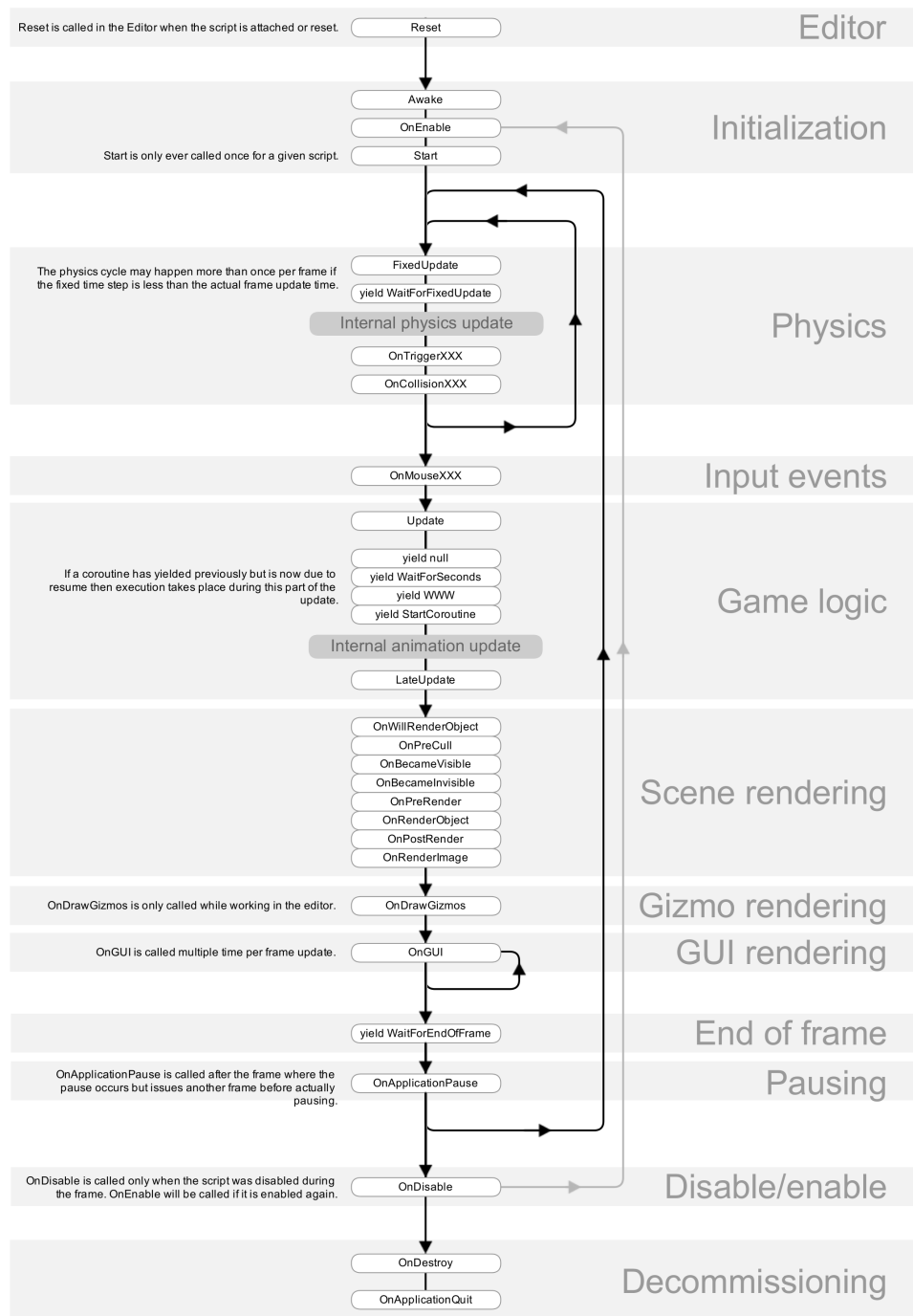


Figura 3.4: Diagramma metodi classe MonoBehaviour [22]

3. BACKGROUND

Quando si crea uno script dal pannello “Project”, lo script eredita in automatico dalla classe **MonoBehaviour**. La classe MonoBehaviour mette a disposizione un framework che permette di collegare gli script a GameObjects all’interno dell’editor, inoltre permette di intercettare determinati eventi relativi al ciclo di vita degli oggetti, in modo da facilitare lo sviluppo delle applicazioni e da eseguire il proprio codice sulla base di quello che sta avvenendo all’interno del progetto.

Di seguito si elencano alcune delle funzioni più importanti della classe **MonoBehaviour**:

- **Awake:** Chiamato quando l’istanza dello script è caricata. È principalmente usato per inizializzare variabili o stati, prima che l’applicazione inizi.
- **Start:** Simile al metodo Awake. Viene chiamato ogni volta che un GameObject inizia ad esistere (quando la scena è caricata, oppure quando il GameObject è inizializzato). A differenza del metodo Awake, il metodo Start non può essere chiamato allo stesso frame di Awake se lo script non è stato attivato a tempo di inizializzazione. Questa temporizzazione tra il metodo Awake e il metodo Start risulta essere molto utile in casi in cui esistono due oggetti A e B e l’oggetto A ha bisogno di utilizzare informazioni contenute nell’oggetto B. In questo caso, l’inizializzazione di B dovrebbe essere fatta in Awake, mentre quella di A in Start.
- **Update:** Chiamato ad ogni frame, se lo script MonoBehaviour risulta attivo. Questo è il metodo più usato per implementare funzionalità di scripting di qualsiasi tipo.
- **FixedUpdate:** Simile al metodo Update, ma è indipendente dal Frame-Rate utilizzato dall’ applicazione. Generalmente, viene chiamato più spesso del metodo Update (ma non in ogni caso!).

- **OnDestroy**: chiamato ogni volta che un `GameObject` viene “distrutto”.
- **StartCoroutine**: Inizia una coroutine. L’esecuzione della coroutine può essere fermata in qualsiasi punto del codice usando lo statement *yield*
- **OnAudioFilterRead**: Unity chiama automaticamente questo metodo ogni volta che un oggetto `AudioSource` cerca di riprodurre un suono. ***OnAudioFilterRead*** riceve un array di floats chiamato ***data*** come parametro, questo array sono i campionamenti PCM che l’oggetto `AudioSource` sta riproducendo. Il secondo parametro ***channels*** rappresenta il numero di canali disponibili che stanno riproducendo il suono.

3.2.5 Audio in Unity

Nella vita reale, i suoni sono generati da oggetti e ascoltati attraverso le orecchie delle persone. Il modo in cui il suono è percepito dipende da molti fattori. Una persona che sta ascoltando un suono riesce a percepire approssimativamente da che direzione e da che lontananza il suono proviene. Un oggetto che si muove velocemente emettendo un suono cambierà di tono mentre si muove, come risultato dell’effetto Doppler. Così come l’ambiente all’interno del quale il suono è stato prodotto cambierà il suono stesso, producendo ad esempio eco.

Per simulare l’effetto della posizione del suono, Unity ha bisogno che i suoni siano generati da una componente **AudioSource**, collegata ad un oggetto in una Scena. I suoni emessi vengono poi percepiti da una componente **AudioListener** collegata ad un altro oggetto (di solito alla camera principale della Scena). In questo modo Unity può simulare l’effetto della distanza e della posizione tra l’oggetto `Audio Source` e l’oggetto `Audio Listener` e riprodurre il suono di conseguenza.

Unity non riesce a calcolare gli echi e gli effetti puramente partendo dalla Scena, ma questi si possono simulare utilizzando **AudioFilters**.

AudioSource: Un oggetto **Audio Source** permette di riprodurre una clip audio all'interno di una Scena. Questa clip, può essere riprodotta verso un Audio Listener, oppure attraverso un Audio Mixer. Un **Audio Source** può riprodurre qualsiasi tipo di clip audio e può essere configurato per riprodurre le clip in modalita 2D, 3D, o una via di mezzo (SpatialBlend).

AudioListener: Un oggetto **Audio Listener** si comporta come se fosse un microfono all'interno di una Scena. Riceve input da ogni Audio Source nella scena e riproduce i suoni attraverso le casse per computer. Solitamente un Audio Listener viene collegato alla camera principale della scena.

AudioFilters: Un oggetto **Audio Filter** permette di modificare l'output di **Audio Source** o di **Audio Listener**, applicando degli effetti audio. Questi possono permettere di filtrare in suono, oppure di applicare riverbero o altri effetti. Gli effetti sono applicati aggiungendo componenti nell'oggetto contenente l'**Audio Source** o l'**Audio Listener** che sta generando il suono. L'ordine in cui queste componenti vengono collegate agli oggetti è importante, in quanto definirà l'ordine in cui gli effetti andranno a modificare l'audio.

3.3 Gesture Recognition

I metodi basilari di interazione in input tra uomo e macchina, sono mouse e tastiera, che non sempre risultano molto pratici agli utenti meno avvezzi all'uso di un computer, o addirittura impossibili per gli utenti portatori di *handicap motori*. Per questo motivo nel nel corso degli anni sono stati studiati metodi alternativi per interagire con un computer.

Negli inizi degli '80, nasce quindi un primo metodo di input alternativo, lo *Speech Recognition*, che permetteva ad un utente di inserire del testo semplicemente parlando. Col passare degli anni lo Speech Recognition è stato ulteriormente sviluppato, permettendo, nei tempi moderni, di eseguire azioni ben più complesse dell'inserimento di semplice testo, come ad esempio impartire il comando di controllare il meteo, effettuare acquisti, o riprodurre

3. BACKGROUND

una playlist musicale.

Nel decennio successivo è stato introdotto un nuovo metodo di input, in grado di interpretare il linguaggio dei segni,...., che diventerà il precursore del *Gesture Recognition*. Il *Gesture Recognition* non si ferma esclusivamente all'interpretazione e riconoscimento del linguaggio dei segni, ma va oltre, andando ad interpretare gesti e movimenti dell'utente, includendo direzione e velocità dei tali.

Al fine di catturare questi movimenti sono state create diverse tipologie di dispositivi, ognuno con diverse caratteristiche, anche se gran parte di questi sono stati sviluppati per un determinato settore, quello videoludico. La prima azienda videoludica che ha fatto uso di questa innovazione è stata la *Nintendo* con il controller *WiiMote* per la console *Nintendo Wii*, una console molto economica ed accessibile ad un mercato molto ampio, la quale ebbe una diffusione notevole grazie alla novità nel settore.

Dato questo enorme successo, anche altre aziende del settore si cimentarono nel seguire questa nuova modalità di controllo: *Microsoft* sviluppò il sensore *Kinect* per la console *Xbox 360* (e successive generazioni, apportando migliorie anche al sensore *Kinect*); la controparte *Sony* sviluppò invece il *Playstation Move*. Il funzionamento di questi 3 controller ha in comune l'utilizzo di telecamere: il *WiiMote* utilizza una telecamera infrarossi sul telecomando, la quale si orienta verso il monitor grazie ad un dispositivo contenente 10 led infrarossi, che funziona da punto cardine; il *Playstation Move* sfrutta una telecamera in grado di riconoscere le sagome umane, ricreando quindi un modello scheletrico di esso, ed usa dei telecomandi specifici per determinare la persona ed i suoi movimenti; il dispositivo *Kinect*, come spiegato nel capitolo precedente, è invece ben più complesso, motivo per il quale ha trovato largo impiego anche al di fuori del mondo videoludico, venendo utilizzato anche in ambito scientifico grazie alla sua elevata efficienza ed il rilascio da parte di *Microsoft* del *Software Development Kit (SKD)*, che permette lo sviluppo di software da terze parti sfruttando il suddetto dispositivo.

Negli anni a venire sono nati sempre più dispositivi che permettessero il *Gesture*

Recognition, soprattutto nel mondo videoludico ed in particolare per il settore della *Virtual Reality*. I più importanti tra questi sono: *Oculus Rift*, *HTC Vive* e, quello da noi selezionato, il *Leap Motion*.



Figura 3.5: WiiMote [24]



Figura 3.6: Kinect [23]

3.4 Leap Motion

Il nostro sistema fa uso di un controller Leap Motion per il riconoscimento dei movimenti delle mani. Si tratta di un piccolo dispositivo collegabile via USB al PC, che utilizza 2 fotocamere IR monocromatiche, insieme a 3 led infrarossi, per osservare un area ad una distanza compresa tra 60 e 80 centimetri, con un campo visivo di $140^\circ \times 120^\circ$. Nello specifico, il device è in grado di riconoscere tutte le 10 dita con una precisione fino a 1×10^3 millimetri. Il software del Leap Motion è in grado di riconoscere 27 elementi distinti della mano, incluse ossa e articolazioni, e tracciarle anche nel caso in cui vengano oscurate da altre parti della mano [20]. Il sensore fornisce tre tipi di informazioni spaziali: (1) la posizione di dita e mani; (2) il vettore movimento delle singole dita; (3) la rappresentazione sferica della curvatura della mani. Queste funzionalità permettono all'utente di interagire con il computer attraverso gesture naturali delle mani, come waving, pinching e swiping. L'azienda UltraLeap fornisce delle API in diversi linguaggi di programmazione, e garantisce l'integrazione con Unity3D.

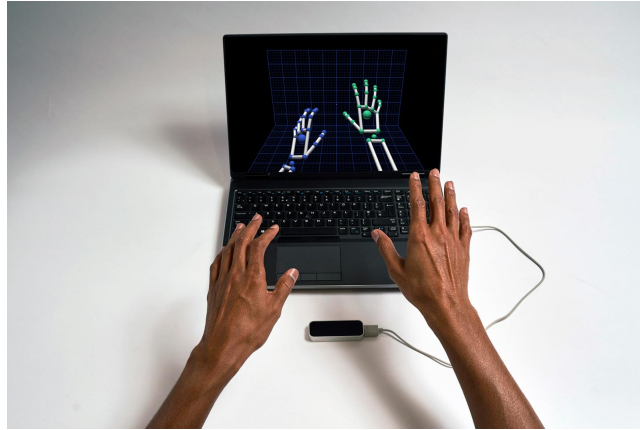


Figura 3.7: Leap Motion Controller

3.5 Artificial Neural Networks

3.5.1 Funzionamento

Le *Reti Neurali Artificiali* (*Artificial Neural Networks*, o *ANN*) sono modelli computazionali ispirati alle reti neurali biologiche, come quelle che compongono il cervello, e di conseguenza sono costituiti da un insieme di interconnessioni di informazioni formate tra neuroni artificiali che elaborano dati e li scambiano l'un l'altro, esattamente come le sinapsi. Ogni neurone è connesso con molti altri, ed i collegamenti possono essere di tipo rafforzativo o inibitorio rispetto l'attivazione degli altri neuroni ad esso connessi. Ognuno di questi neuroni usa una funzione apposita per combinare tutti i valori di input con una *funzione di attivazione*, che elabora e restituisce l'output del neurone:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

Dove w_i è la matrice dei pesi (**weights**) assegnati a ciascun input, che viene aggiunta ad un termine b (**bias**). I due insiemi **weights** e **bias** vengono generati nella fase di addestramento, e saranno conservati per l'utilizzo. Tramite questa funzione, la rete neurale filtra gli input, operando solo quelli che rientrano in un determinato limite imposto. Una rete neurale è spesso di tipo *adattivo*, ovvero che “muta” la propria struttura in base a modelli

ricevuti in input durante la *fase di apprendimento*.

3.5.2 Ottimizzazione

Per ottimizzare la funzione di attivazione, è possibile introdurre una soglia, che servirà a decrementare il valore d'ingresso della funzione. Il metodo più diffuso è l'utilizzo di una delle seguenti funzioni:

- Funzione di **Sigmoid**;
- Funzione di **TanH** (tangente iperbolica);
- Funzione di **ReLU** (rectified linear unit);

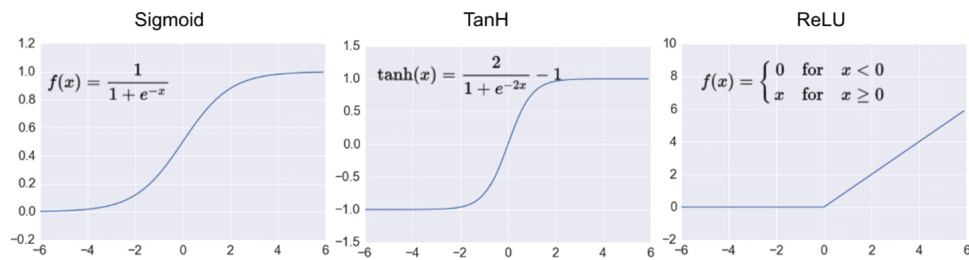


Figura 3.8: [25]

3.5.3 Struttura di una rete neurale

La struttura di una rete neurale può essere divisa in tre livelli, ognuno dei quali ha un numero distinto di neuroni:

- **Livello di input:** riceve ed elabora i segnali ricevuti in ingresso, inoltrandoli al **livello nascosto**;
- **Livello nascosto:** vengono effettuati tutte le elaborazioni necessarie per il training sui dati ricevuti dal **livello di input**;
- **Livello di output:** ultimo livello, dove vengono raccolti tutti i dati uscenti dal livello nascosto;

Uno dei difetti delle reti neurali, nonostante l'elevata efficienza, è la non spiegabilità delle decisioni avvenute al loro interno, difatti questi modelli sono chiamati “*Black Box*”.

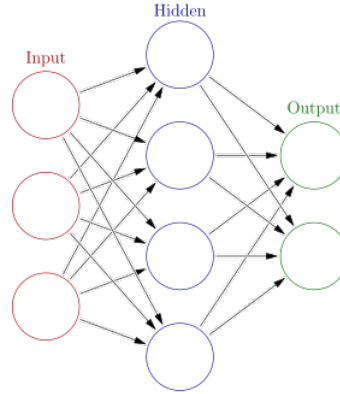


Figura 3.9: Rete Neurale Artificiale [26]

3.6 Paradigmi di apprendimento

L'apprendimento può essere effettuato seguendo tre tipi diversi di paradigmi: *apprendimento supervisionato*, *apprendimento non supervisionato* ed *apprendimento per rinforzo*.

- **Apprendimento supervisionato (*supervised learning*):**

Quando si ha a disposizione un insieme di dati, anche chiamato *training set*, contenente esempi d'ingresso con il relativo output, la rete imparerà ad stabilire la relazione che collega tutti i dati del *training set*.

Nella fase di addestramento, vengono utilizzati appositi algoritmi come il **backpropagation**, che ciclicamente sfrutta i dati ricevuti in input per stimare i *weight* ed i *bias* della rete di neuroni in modo tale da minimizzare al meglio l'errore di riconoscimento del *training set*. Se l'addestramento è stato effettuato con successo, quindi con una *accuracy* elevata, la rete ha imparato a riconoscere la relazione che lega i dati in ingresso con quelli in uscita, e sarà quindi in grado di effettuare previsioni anche su dati differenti da quelli del *training set*, ovvero su qualsiasi dato (purchè rispetti la forma di input accettata) ricevuto in ingresso. La rete

3. BACKGROUND

sarà quindi in grado di risolvere problemi di classificazione e potrà essere chiamata *classifier*.

- **Apprendimento non supervisionato (*unsupervised learning*):**

Metodo di apprendimento basato su algoritmi che riclassificano e organizzano i dati passati in input in base a caratteristiche comuni tra di loro, per effettuare previsioni sugli input ricevuti successivamente. a differenza dell'apprendimento supervisionato, i dati ricevuti in fase di addestramento non sono annotati, dato che le classi da riconoscere non sono conosciute inizialmente, ma verranno distinte durante l'addestramento stesso.

- **Apprendimento per rinforzo (*reinforcement learning*):**

Questo metodo di apprendimento, a differenza dei due precedenti, opera come una automa deterministico, occupandosi di problemi di decisioni sequenziali, dove l'output dipende esclusivamente dallo stato in cui si trova il sistema determinandone il successivo. La qualità delle decisioni dipende da un valore chiamato *ricompensa*, maggiore è la ricompensa e più propenso sarà per prendere quella decisione. Le *ricompense* possono essere sia positive che negative, in modo tale da incoraggiare o scoraggiare determinate scelte.

La scelta del paradigma dipende dal tipo di ecosistema che si vuole creare, e dal tipo di dati di cui si è in possesso. Se si dispone di dati da fornire in input, e si conosce l'output atteso (detto **label**), la scelta ricade automaticamente sull'*apprendimento supervisionato*. Se invece non si sa l'output desiderato a priori ma si dispone di un dataset, la scelta sarà l'*apprendimento non supervisionato*. In fine, nel caso si volesse creare un sistema in grado di prendere decisioni da sè, l'opzione migliore è l'**apprendimento per rinforzo**.

3.7 Fondamenti di Acustica

Il suono viene creato da turbolenze ondulatorie emesse da una “sorgente sonora” e propagate nell’ambiente circostante (solitamente l’aria). Si tratta di onde di pressione che vengono percepite come suoni dall’orecchio umano.

Le onde sonore possono essere rappresentate graficamente utilizzando un grafico cartesiano, riportante il tempo (t) sull’asse delle ascisse, e gli spostamenti delle particelle (s) su quello delle ordinate. Il tracciato esemplifica gli spostamenti delle particelle: alla fine, la particella si sposta dal suo punto di riposo (asse delle ascisse) fino al culmine del movimento oscillatorio, rappresentato dal ramo crescente di parabola che giunge al punto di massimo parabolico. Poi la particella inizia un nuovo spostamento in direzione opposta, passando per il punto di riposo e continuando per inerzia fino ad un nuovo culmine simmetrico al precedente, questo movimento è rappresentato dal ramo decrescente che, intersecando l’asse delle ascisse, prosegue in fase negativa fino al minimo parabolico. Infine, la particella ritorna indietro e ripete nuovamente la sequenza di spostamenti, così come fa il tracciato del grafico.

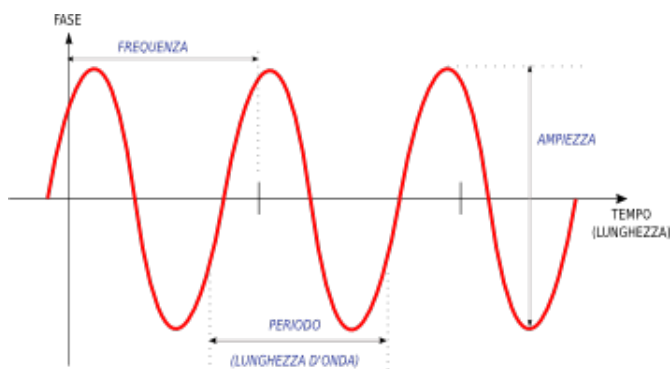


Figura 3.10: Rappresentazione di una Sinusoide

Il periodo (graficamente il segmento tra due creste) è il tempo impiegato dalla particella per tornare nello stesso punto dopo aver cominciato lo spostamento (indica cioè la durata di una oscillazione completa). La distanza dalla cresta all’asse delle ascisse indica, invece, l’ampiezza del movimento.

In acustica, si preferisce solitamente usare altre grandezze derivat da queste ultime. Il numero di periodi compiuti in un secondo esprime la frequenza in hertz (Hz). Dall'ampiezza dell'onda, invece, si calcola la pressione sonora, definita come la variazione di pressione rispetto alla condizione di quiete, e la potenza e l'intensità acustica, definita come il rapporto tra la potenza dell'onda e la superficie da essa attraversata; l'intensità delle onde sonore viene comunemente misurata in decibel.

3.8 Digital Audio

Il suono viene creato da turbolenze ondulatorie emesse da una “sorgente sonora” e propagate nell'ambiente circostante (solitamente l'aria). Si tratta di onde di pressione che vengono percepite come suoni dall'orecchio umano.

Per convertire un'onda sonora analogica in un segnale digitale, il computer deve essere in grado di misurarne l'ampiezza in determinati punti. Ogni misura rilevata viene chiamata *campione*, perciò la conversione da analogico a formato digitale prende il nome “campionamento del suono”.

Il teorema del campionamento di Nyquist-Shannon, afferma che se la frequenza di campionamento è sufficientemente grande, non si hanno perdite di informazione rispetto alla forma d'onda originale. Dunque, più ampia sarà la frequenza di campionamento, tanto maggiore sarà anche il numero delle misurazioni d'ampiezza e, quindi, la precisione del segnale digitale.

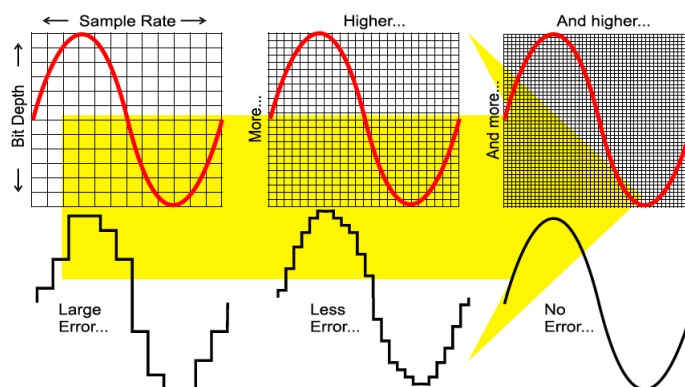


Figura 3.11: Margine di errore per frequenze di campionamento crescenti [27]

Se viene usata una frequenza di campionamento alta, la serie di numeri prodotta contiene pressoché intatta tutta la informazione sulla forma d'onda elettrica analogica iniziale. Nei moderni standard tecnologici, in genere le frequenze di campionamento spaziano dagli 8.000 campioni al secondo (Samples per second, S/s) per la voce telefonica, fino ai 44.100 e più campioni al secondo per la qualità musicale. Queste letture di valori di tensione possono poi cadere in un qualsiasi punto della dinamica del segnale, cioè ogni singolo campione può avere un valore compreso tra il minimo e il massimo possibile. Siccome, potenzialmente si possono avere infiniti valori di lettura di tensione per ogni singolo campione. Per completare l'opera di conversione del segnale da analogico in digitale, va ora suddivisa tutto il possibile range dinamico del segnale in un numero finito di intervalli e ogni singolo intervallo va codificato con un valore digitale ben determinato.

Queste due operazioni si chiamano quantizzazione e codifica di sorgente. La quantizzazione in genere suddivide il range dinamico del segnale in un numero di intervalli potenza del due (2^n intervalli), in maniera tale che ogni singolo campione cadrà inevitabilmente in uno degli intervalli quantizzati e potrà così essere codificato digitalmente con n bit. I valori più ricorrenti di digitalizzazione attualmente usati vanno da un minimo di 8 bit per campione in campo telefonico (range dinamico del segnale suddiviso in 256 intervalli), fino a 20 e più bit per campione (range dinamico del segnale suddiviso in un milione e più di intervalli).

CAPITOLO 4

MARCOSMILES 2.0

L'obiettivo principale di MarcoSmiles 2.0 è quello di superare i limiti della sua precedente version: MarcoSmiles [5]. L'idea è quella di permettere all'utente di poter realizzare delle performance musicali in maniera più naturale e performante. Il sistema utilizza esclusivamente informazioni contenute nella configurazione delle mani, e associa tali configurazioni a note musicali in real-time. È importante ricordare che il sistema non dispone di una configurazione predefinita. Sarà compito dell'utente creare l'astrazione dello strumento addestrandolo a riconoscere configurazioni delle mani secondo le sue preferenze. Nella sua versione precedente, il lavoro è stato svolto interamente utilizzando il linguaggio di programmazione Java. Durante lo sviluppo di questo lavoro, tuttavia, sono emerse diverse diverse problematiche in merito alla reattività dell'interfaccia, e sulle performance in generale, imputabili alla scelta di Java come linguaggio su cui basare il sistema. Inoltre anche la gestione della generazione del suono real-time è risultata particolarmente difficoltosa, motivo per cui, il progetto è stato accantonato. Sulla base di queste esperienze abbiamo deciso di riprogettare il sistema. La scelta sulle tecnologie da usare è ricaduta su Unity3D e C# per quanto riguarda l'interfaccia grafica e le funzioni di base, mentre per la parte di Machine Learning, abbiamo usato

la libreria SciKit Learn in Python. Il sistema da noi sviluppato è dunque strutturato in una componente hardware, e una componente software. La componente hardware è rappresentata dal Leap Motion Controller, che può essere posizionato a piacimento dell'utente, solitamente su una scrivania con i sensori rivolti verso l'alto. La componente software consiste nel sistema che effettua l'associazione configurazione - nota musicale in real-time. In fase di progettazione abbiamo suddiviso la parte software in 3 moduli:

1. **Modulo Unity:** Interfacciamento Leap Motion Controller e GUI;
2. **Modulo Machine Learning:** Registrazione dati, elaborazione e testing
3. **Modulo Sonoro:** Generazione audio real-time & Pulse-Code Modulation

Il lavoro di tesi in oggetto riguarda il primo modulo del sistema. In particolare verranno descritte le principali funzionalità del sistema, la costruzione e la progettazione dell'interfaccia grafica e del dataset, le features selezionate per il machine learning, e la gestione delle configurazioni del sistema.

CAPITOLO 5

IL NOSTRO APPROCCIO

5.1 Costruzione del dataset

La scelta dei dati da utilizzare nel dataset è di fondamentale importanza nell'ambito di un problema di classificazione, infatti l'output della ANN dipende in gran parte dalla scelta effettuata. Per ogni sample catturato dal LeapMotion bisogna considerare un adeguato insieme di features da utilizzare per la gesture recognition. Una soluzione consisterebbe nell'utilizzare i "raw data" acquisiti dal Leap Motion come insieme di features. Tuttavia questo approccio non permette di ottenere buoni risultati visto che i dati sono dipendenti da numerosi fattori quali la posizione, l'orientamento e la dimensione delle mani. Dunque anche un piccolo movimento produrrebbe una minor precisione in fase di classificazione. Nell'effettuare la scelta dei dati da utilizzare abbiamo considerato lo studio effettuato nella precedente versione del sistema MarcoSmiles. In questo tipo di problema la letteratura suggerisce di adottare un insieme di features indipendente dai suddetti fattori. Utilizzando questo approccio, una gesture è tale indipendentemente dalla traslazione, rotazione e dimensione della mano. Ciò significa che il sistema è in grado di riconoscere la gesture senza tener conto dei fattori sopracitati.

5. IL NOSTRO APPROCCIO

Le API fornite dal Leap Motion permettono di ottenere e manipolare diverse informazioni circa le mani. Nel caso del sistema *MarcoSmiles*, sappiamo esattamente la classe di input e la *label*, abbiamo quindi selezionato il metodo di apprendimento supervisionato. La struttura del nostro dataset sarà la seguente:

$$(C_{LH}, C_{RH}, id)$$

Dove C_{LH} e C_{RH} sono configurazioni di dati relativamente per la mano sinistra e la mano destra, mentre id sarà la *label* corrispondente alla nota musicale che si desidera suonare.

Ogni configurazione delle mani è composta da diverse combinazioni delle *features* da noi selezionate:

- **Nearest Finger Angle (NFA)**: siano f_1, f_2 due dita tali che $f_1, f_2 \in S_{LH}$ o $f_1, f_2 \in S_{RH}$, l'NFA è l'angolo θ tra f_1 ed f_2 ed è calcolato come l'angolo tra il vettore-direzione di f_1 ed il vettore-direzione di f_2 (vedi Figura).
- **Finger Flexion (FF)**: sia $f \in S_{LH} \cup S_{RH}$ un dito, la *flessione* di f è l'angolo θ tra i vettori direzione associati al metacarpo ed alle ossa distali di f (vedi Figura).

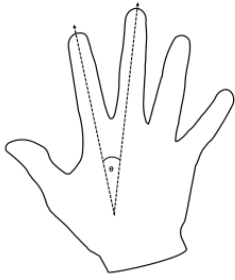


Figura 5.1: Nearest Fingers Angles (NFA)

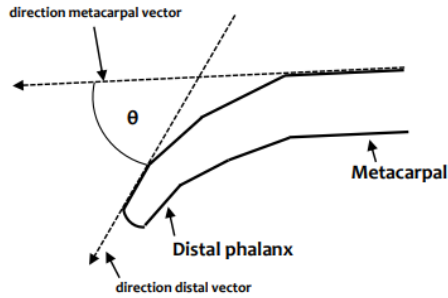


Figura 5.2: Finger Flexion (FF)

Denotiamo con l_5, l_4, l_3, l_2, l_1 le dita della mano sinistra da mignolo a pollice, mentre denotiamo con r_1, r_2, r_3, r_4, r_5 le dita della mano destra, da pollice a mignolo. Inoltre indichiamo con $S_{LH} = \{l_5, l_4, l_3, l_2, l_1\}$ l'insieme delle dita

della mano sinistra, e con $S_{RH} = \{r_1, r_2, r_3, r_4, r_5\}$ l'insieme delle dita della mano destra.

Le combinazioni di *features* presenti nelle configurazioni C_{LH} e C_{RH} sono le seguenti:

- C_{LH} :

$$FF_0(l_1), FF_1(l_2), FF_2(l_3), FF_3(l_4), FF_4(l_5), \\ NFA_0(l_0, l_1), NFA_1(l_1, l_2), NFA_2(l_2, l_3), NFA_3(l_3, l_4)$$

- C_{RH} :

$$FF_0(r_1), FF_1(r_2), FF_2(r_3), FF_3(r_4), FF_4(r_5), \\ NFA_0(r_0, r_1), NFA_1(r_1, r_2), NFA_2(r_2, r_3), NFA_3(r_3, r_4)$$

Nei *training set* utilizzati per l'addestramento ad ogni nota corrispondono 500 combinazioni di (C_{LH}, C_{RH}, id) , dove l'elemento *id* è comune a tutte le 500. Ognuna di queste combinazioni varierà leggermente dalle altre, dato che la registrazione del dataset è stata effettuata salvando la posizione delle mani nell'arco di due minuti per ogni nota, durante i quali si presentano leggere variazioni, volontarie e non. Queste variazioni volontarie sono effettuate al fine di fornire più informazioni all'algoritmo di apprendimento, dato che in una situazione reale la possibilità di riposizionare le mani nella esatta posizione mantenuta durante la fase di addestramento è scarsa.

5.2 Compatibilità delle librerie

Per quanto riguarda l'implementazione del modulo di *Machine Learning* sono state riscontrate diverse difficoltà, di cui tratterà questo paragrafo. Le librerie selezionate nel corso dello sviluppo sono state le seguenti:

- Unity ML-Agents;
- Microsoft ML.Net;
- Python Scikit-Learn;

Inizialmente, essendo il sistema *MarcoSmiles* progettato sulla piattaforma *UnityEngine*, l'idea era di sfruttare la libreria da esso offerta, ovvero **ML-Agents**.

La libreria **ML-Agents** è una libreria open-source che permette di creare ambienti per il training di *agenti intelligenti*, ideale quindi sia per videogiochi che per ricerche sulle intelligenze artificiali. La comodità di questa libreria è l'integrazione diretta all'interno del progetto, è sufficiente infatti importarla all'interno di esso. Per quanto riguarda l'apprendimento, invece, è necessario installare Python e determinate librerie. Purtroppo il metodo di apprendimento a noi necessario non era supportato, ed abbiamo quindi deciso di scartare questa libreria, passando alla successiva.

La libreria di casa **Microsoft**, **ML.Net** offre altrettante funzionalità ed è orientata verso un ambiente più generico e non solo a Unity Engine. Essendo implementata nel framework .NET essa è utilizzabile, oltre che nell'ecosistema *Windows*, anche sui sistemi operativi *Linux* e *MacOS*. Questa libreria sta riscontrando un enorme successo, in particolare per i software che sfruttano il framework .NET, creando motori di machine learning integrati nel software. I punti di forza di questa libreria sono:

- La disponibilità per i sistemi operativi più diffusi;
- Vasta scelta di modelli di Machine Learning;
- Possibile integrazione con *Microsoft Azure*;

Purtroppo la versione del framework supportato da Unity non rientra nella lista dei framework supportati da questa libreria, rendendola automaticamente inutilizzabile per il nostro progetto.

L'ultima libreria che andremo ad analizzare è quella di **Python**, **Scikit-Learn**. Questa libreria è considerata la libreria di riferimento per il machine learning open-source e non. Grazie alle sue funzionalità ed al suo particolarmente semplice utilizzo ha una versatilità eccellente, che

permette di utilizzarla in contesti molto vasti. Nonostante la semplicità d'uso, l'integrazione non è stata altrettanto intuitiva. Essendo i progetti unity scritti interamente nel linguaggio di programmazione C#, il linguaggio python non è direttamente supportato. È stato quindi necessario creare un *wrapper*, all'interno del nostro programma. Un wrapper consente di eseguire programmi o script di codice esterni al programma principale, nel nostro caso progettato in C#. Questo wrapper ci permette quindi di eseguire il codice python necessario per eseguire l'apprendimento supervisionato sul dataset semplicemente facendo click su un pulsante. Il wrapper aprirà una finestra esterna, sulla quale compariranno gli output dello script python, i quali mostreranno lo stato corrente del learning.

5.3 Classificatore Machine Learning

Come spiegato nel capitolo precedente, nel lavoro svolto in questa tesi le reti neurali saranno addestrate per creare una classificazione di dati inerenti alle determinate configurazioni studiate nel paragrafo 5.1, al fine di poter riconoscere posizioni specifiche delle mani. Il riconoscimento di tali posizioni non è mai un ruolo semplice, e spesso la correttezza dei dati può essere lesa da diversi fattori, come un dispositivo di cattura non totalmente funzionante, malfunzionamenti del calcolatore o addirittura un errore nei modelli utilizzati. Con l'estrazione delle *features* selezionate è possibile classificare delle posizioni mai incontrate prima dal sistema, attribuendogli una classe di appartenenza (la *label*) che nel nostro sistema corrisponde ad una nota musicale da riprodurre digitalmente in tempo reale. Il paradigma di apprendimento selezionato in questa tesi sarà *l'apprendimento supervisionato* spiegato nel capitolo precedente.

La giusta interpretazione delle features è un elemento fondamentale per la classificazione di dati, e sancisce totalmente l'esito della classificazione. Come già spiegato, per costruire il nostro classificatore, verranno prese in considerazione solo determinati dati inerenti alle mani dell'utente, tutte

rientranti nella configurazione (C_{LH}, C_{RH}, id) .

5.3.1 Albero decisionale

Un albero decisionale (o *Decision Tree*) è un grafo di predizione composto da n variabili input ed m variabili output, i quali corrispondono rispettivamente a *features* ed alle *decisioni*, le ultime variabili in output rappresentano invece le *decisioni finali* e quindi le azioni che verranno intraprese. Quando un albero riceve un input, la decisione segue un percorso dall'alto al basso, attraversando i cosiddetti *rami*. Ogni ramo termina su un **nodo**, il quale verifica una condizione su una *feature*. I nodi possono essere di tre tipi:

- **Nodo Radice:** è il primo nodo di un albero, non ha rami entranti.
- **Nodo Interno:** quando possiede diramazioni verso il basso.
- **Nodo Foglia:** quando non possiede diramazioni, ed è quindi un *nodo terminale*

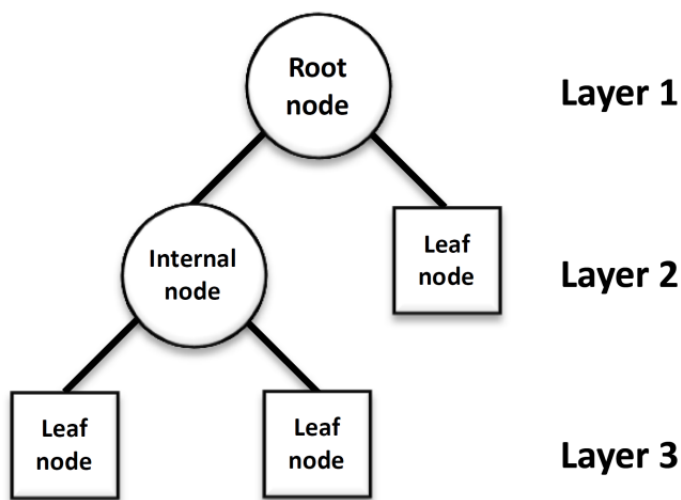


Figura 5.3: Albero Decisionale [33]

5.3.2 Random Forest

Nel machine learning, una **foresta casuale** è un *classificatore d'insieme*, ed è legato all'apprendimento supervisionato. Fa uso di *bagging*, per creare l'aggregazione di alberi decisionali. Essendo un modello di *ensemble* fa uso di diversi algoritmi di apprendimento, per combinare le diverse previsioni, al fine di generare previsioni più accurate rispetto ai modelli che operano come singoli. In questo modo, la varianza viene ridotta drasticamente. L'importanza di una varianza più bassa possibile è fondamentale, dato che gli alberi decisionali dipendono sensibilmente ai dati sui quali sono composti. Anche con una semplice modifica di questi dati potrebbe portare ad alberi differenti, e quindi previsioni anche totalmente contrastanti. Le previsioni di un albero individuale possono quindi essere di scarsa efficienza, ma il modello di *random forest*, combinando le previsioni di diversi alberi decisionali, riesce ad ottenere previsioni più accurate e costanti. In un contesto di classificazione, il risultato di una *random forest* equivale quindi alla classe output restituita dal maggior numero di alberi.

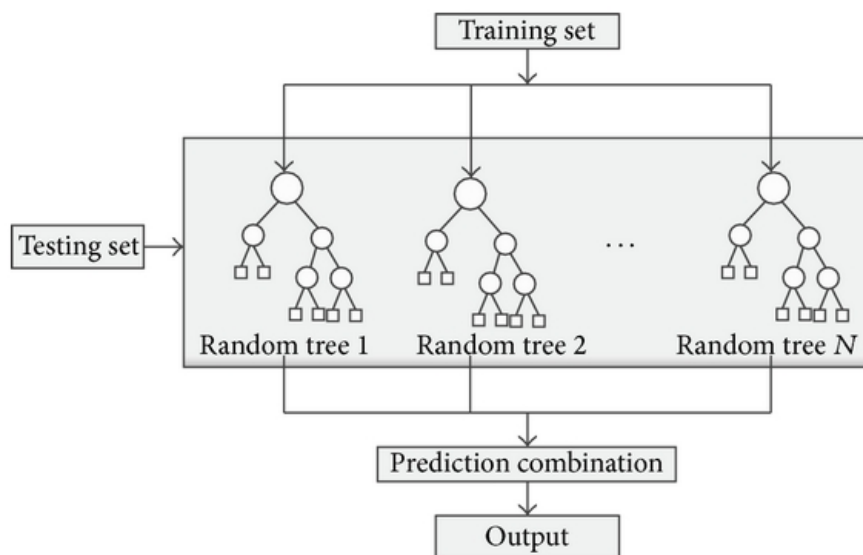


Figura 5.4: Esempio di Random Forest [34]

5.4 Feature Scaling

Nel machine learning, ed in particolare nell'apprendimento supervisionato come nel nostro caso, non è raro il confronto tra differenti unità di misura ed una successiva decisione. Se ad esempio si è in possesso di un dataset con due sole caratteristiche con ordini di misura differenti, la caratteristica con i valori più elevati potrebbe influenzare il risultato in maniera maggiore rispetto alla seconda caratteristica, proprio a causa del suo valore largamente maggiore. Questo però non significa per forza che questa caratteristica sia più importante dell'altra, andando a compromettere le prestazioni del classificatore. Un classificatore però non sa a priori a quale caratteristica dare maggiore priorità, quindi il bilanciamento di questi valori va gestito prima dell'addestramento del classificatore. La tecnica più diffusa per questo scopo è chiamata *features scaling*, che consiste nel ridimensionare (o bilanciare) i dati di studio, riuscendo quindi ad uguagliare l'importanza dei dati, al fine di compararli indipendentemente dall'ordine di grandezza.

“Il feature scaling è un metodo utilizzato per standardizzare l'intervallo di variabili indipendenti o caratteristiche dei dati. Nell'elaborazione dei dati, è anche nota come normalizzazione dei dati e viene generalmente eseguita durante la pre-elaborazione dei dati.”[21]

Con il termine scaling si intende un'operazione di addizione o sottrazione di un valore costante, seguita da un'operazione di moltiplicazione o divisione per un altro valore costante, in modo tale che le features si possano trovare entro determinati valori di minimo e di massimo.

Questo ridimensionamento consente di gestire anche le deviazioni più piccole delle funzionalità. Generalmente quando si utilizza un intervallo tra 0 e 1 si effettua una **standardizzazione**.

Vi sono diversi metodi di scaling, ma tre sono i più diffusi:

- **Standardizzazione Z-Score**
- **Normalizzazione MinMax Scaling**
- **Normalizzazione media**

La **standardizzazione Z-Score** consiste nel ridimensionamento degli attributi in modo tale che il valore medio μ sia 0 e la deviazione σ sia 1. Si otterrà quindi che le caratteristiche sono ridimensionate in maniera tale da avere le proprietà di una distribuzione normale con media nulla e con deviazione standard di 1. Per calcolare il valore di Z-Standard si usa la seguente formula:

$$Z = \frac{x_i - \mu}{\sigma}$$

Dove μ è la *media* dei valori delle features, σ è la *deviazione standard* delle features e x_i è il valore della feature che si vuole *standardizzare*. Questa operazione viene eseguita su ogni feature che verrà utilizzata nella fase di addestramento. I valori di μ e di σ saranno quindi salvati per poi essere utilizzati sui dati successivi.

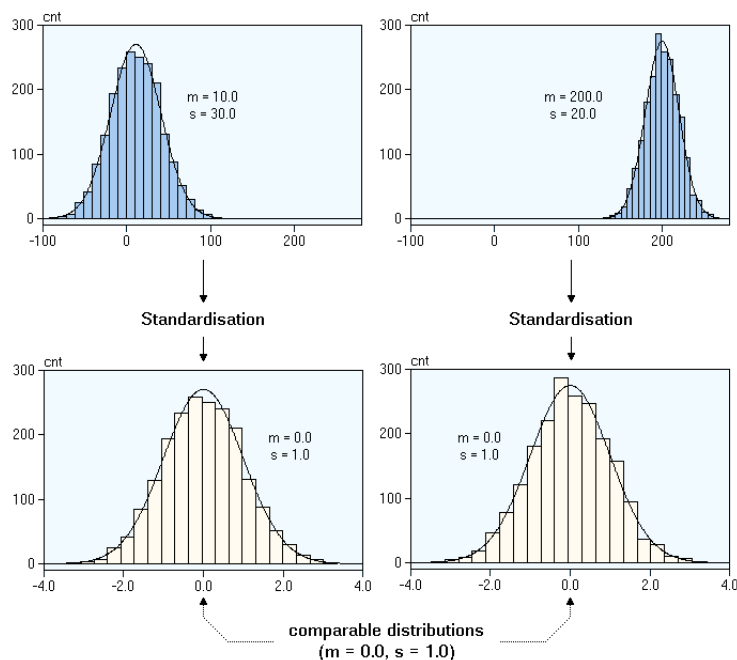


Figura 5.5: Standardizzazione Z-Score

5. IL NOSTRO APPROCCIO

La **Normalizzazione MinMax Scaling** è un'alternativa alla standardizzazione Z-Score preferibile quando non si è a conoscenza della distribuzione dei dati o quando si sa che la distribuzione non è gaussiana. In questo tipo di scaling i dati vengono ridimensionati entro un intervallo fisso, genericamente tra 0 e 1. Per calcolare il valore si usa la seguente formula:

$$Z = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

Il metodo della **normalizzazione MinMax** rende i dati più adatti per operazioni di comparazione o convergenza. Con questo metodo si minimizza la massima perdita possibile massimizzando al contempo il guadagno minimo.

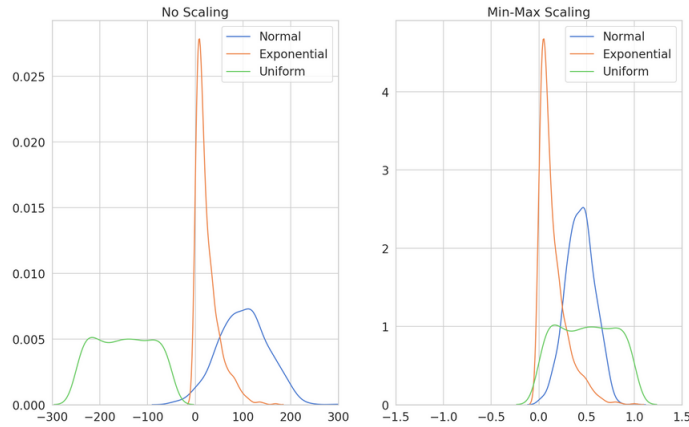


Figura 5.6: Normalizzazione MinMax [32]

La **Normalizzazione media** è molto simile alla normalizzazione MinMax, l'unica differenza nell'equazione è al numeratore, dove anziché del valore minimo vi è il valore medio, come nella seguente formula:

$$Z = \frac{x_i - \text{media}(x_i)}{\max(x_i) - \min(x_i)}$$

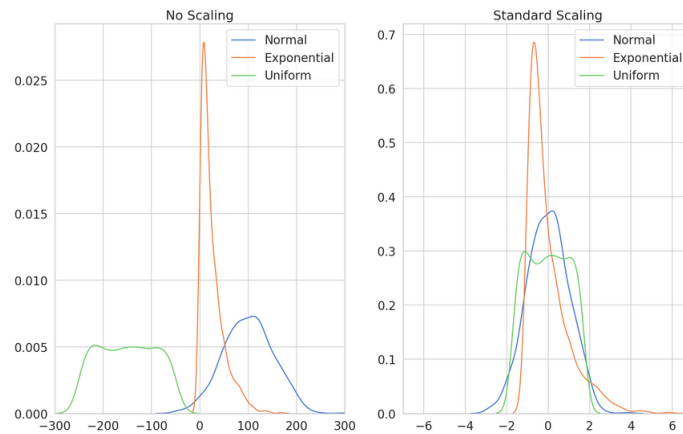


Figura 5.7: Normalizzazione Media [32]

5.5 Strategie Utilizzate

Nei paragrafi precedenti sono state discusse metodologie e funzionamenti dedicate al machine learning, in questo paragrafo saranno trattate quindi le scelte tra le diverse possibilità offerte. Ci sono differenti scelte da effettuare riguardo:

1. **Formazione del dataset;**
2. **Paradigma di Apprendimento;**
3. **Script di Machine Learning;**
4. **Metodo di scaling;**
5. **Funzione di attivazione;**
6. **Feature Recognition;**

5.5.1 Formazione del dataset

Per un addestramento ottimale c'è bisogno di una grande mole di dati, in modo da fornire più informazioni possibili alla rete neurale da addestrare. Più dati si hanno a disposizione e più facilmente si riusciranno a riconoscere situazioni inerenti alla classe di dati ricevuta in input. Il nostro

5. IL NOSTRO APPROCCIO

dataset è quindi composto da 500 righe di testo per ogni nota. Queste righe di testo conterranno dati riguardo le configurazioni spiegate nel paragrafo 5.1 separate da una virgola. Ad ogni dato equivale un valore decimale ottenuto mediante funzione in fase di registrazione. È qui riportato un esempio formato da 3 righe presenti in un dataset di training:

```

_1 59.40927, 148.4462, 147.538, 147.6497, 149.3008, 152.574,
2.303078, 9.072443, 10.93387, 50.51312, 152.8297, 149.8195,
150.2057, 150.0309, 149.3782, 3.920764, 9.012863, 9.368274, 0

_2 62.33798, 146.9624, 146.4716, 146.4662, 148.1802, 153.0986,
1.903823, 9.134305, 11.10348, 52.3268, 152.0411, 148.4747,
149.1424, 149.3762, 150.7578, 4.702696, 8.933953, 9.566121, 0

_3 61.4049, 146.6705, 145.6753, 145.5838, 147.2906, 153.1232,
1.755243, 9.343422, 10.85496, 50.51425, 152.7829, 149.1697,
149.6153, 149.4406, 150.7445, 4.826088, 8.815902, 9.580413, 0
```

Un dataset ideale è formato da almeno 12 note, in modo tale da coprire un’ottava intera, e comprenderà quindi un minimo di 6000 righe contenenti informazioni utili per l’addestramento della rete neurale. Attualmente, il numero massimo di note registrabili è di 24, ovvero due ottave, ma la dimensione del dataset non è limitata a sole 500 righe per ogni nota, è infatti possibile registrare ulteriori dati per una stessa nota, in modo tale da poter ottenere un potenziale miglior riconoscimento per tale configurazione. Aumentare la dimensione del dataset, comporta però un aumento di tempo necessario per quanto riguarda l’addestramento della rete, come vedremo nel capitolo 6.

5.5.2 Paradigma di Apprendimento

Una volta creato il dataset, se lo si analizza ed interpreta, si può vedere il paradigma di apprendimento ideale per esso. Nel nostro caso il dataset contiene sia le *features* che la *label*, entrambe caratteristiche necessarie per il paradigma di *apprendimento supervisionato*, come spiegato nel paragrafo 3.6. La nostra rete neurale è stata quindi addestrata tramite apprendimento supervisionato.

5.5.3 Script di Machine Learning

In questo paragrafo sarà mostrato lo script utilizzato per l'addestramento della rete neurale.

Copme già accennato, faremo uso della libreria SciKit-Learn, la quale è implementata esclusivamente per il linguaggio python. Di seguito è riportato lo script utilizzato per effettuare il training.

Listing 5.1: Script Machine Learning

```
1 import os
2 import numpy as np
3 from sklearn.metrics import precision_recall_fscore_support as score
4 from sklearn.preprocessing import RobustScaler, StandardScaler, MinMaxScaler
5 from sklearn.preprocessing import LabelEncoder
6 from sklearn.model_selection import GridSearchCV
7 from numpy import mean
8 from sklearn.metrics import accuracy_score
9 from sklearn.model_selection import train_test_split
10 from sklearn.neural_network import MLPClassifier
11 import pickle
12 from sklearn.ensemble import ExtraTreesClassifier
13 import matplotlib.pyplot as plt
14
15 import pandas as pd
16 from _datetime import datetime
17
18 # CREAZIONE FILE PER REGISTRARE LA DURATA E SCRITTURA INIZIO APPRENDIMENTO
19 with open('learning_time.txt', 'w') as lt:
```

5. IL NOSTRO APPROCCIO

```
20     now = datetime.now()
21     # dd/mm/YY H:M:S
22     dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
23     lt.write(str("Data Inizio Learning: " + dt_string + "\n"))
24
25 # FUNZIONE PER LA SCRITTURA DELLA FEATURE IMPORTANCE
26 def feature_importance(X, y):
27     # Build a forest and compute the impurity-based feature importances
28     forest = ExtraTreesClassifier(n_estimators=250, random_state=0)
29
30     forest.fit(X, y.values.ravel())
31     importances = forest.feature_importances_
32     std = np.std([tree.feature_importances_ for tree in forest.estimators_], axis=0)
33     indices = np.argsort(importances)[-1:]
34
35     # Print the feature ranking
36     print("Feature ranking:")
37
38     for f in range(X.shape[1]):
39         print("%d. feature %d (%f)" % (f + 1, indices[f], importances[indices[f]]))
40
41     # Plot the impurity-based feature importances of the forest
42     plt.figure()
43     plt.title("Feature importances")
44     plt.bar(range(X.shape[1]), importances[indices], color="r", yerr=std[indices],
45            align="center")
46     plt.xticks(range(X.shape[1]), indices)
47     plt.xlim([-1, X.shape[1]])
48     plt.savefig('foo.pdf')
49
50
51 # FUNZIONE PER L'ADDESTRAMENTO
52 def grid(cl, classifier, param_grid, n_folds, t_s_D, tLab_downsampled):
53     with open(cl+".out.txt", "w") as f:
54
55         # sceglie quanti core della cpu utilizzare, verranno usati solo
56         # 5/6 di tutti i core disponibili
```

```
57     quotient = 5/6
58     n_core = int(quotient * os.cpu_count())
59
60     estimator = GridSearchCV(
61         classifier, cv=n_folds, param_grid=param_grid, n_jobs=n_core,
62         verbose=1, scoring='f1_weighted')
63     estimator.fit(t_s_D, tLab_downsampled)
64     means = estimator.cv_results_['mean_test_score']
65     stds = estimator.cv_results_['std_test_score']
66     best = []
67     max = 0
68     for meana, std, params in zip(means, stds,
69         estimator.cv_results_['params']):
70         if meana > max:
71             max = meana
72             best = params
73             print("%0.3f (+/-%0.03f) for %r" % (meana, std * 2, params))
74             f.write("%0.3f (+/-%0.03f) for %r" % (meana, std * 2, params))
75             f.write("\n")
76     print()
77     return best
78
79
80 # LETTURA E CARICAMENTO DEL DATASET
81 dataset_array = pd.read_csv('marcosmiles_dataset.csv', sep=',', header=None)
82 x = dataset_array.copy()
83 y = dataset_array.copy()
84
85 xcols = [18]
86 ycols = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17]
87 x.drop(x.columns[xcols], axis=1, inplace=True)
88 y.drop(y.columns[ycols], axis=1, inplace=True)
89
90 # DIVISIONE STRATIFICATA
91 # Divido il dataset in modo stratificato in una parte per il training (80%)
92 # ed una parte per il testing (20%)
93 training_set_data, test_set_data, training_set_labels, test_set_labels =
```

5. IL NOSTRO APPROCCIO

```
94     train_test_split(x, y, test_size=0.2, stratify=y)
95
96 # SCALING
97 # scaling dei dati tramite Normalizzazione MinMax
98 scaler = MinMaxScaler()
99 train_scaled_D = scaler.fit_transform(training_set_data)
100 test_scaled_D = scaler.transform(test_set_data)
101
102 ycols = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17]
103
104 min_values = []
105 max_values = []
106
107 for col in ycols:
108     min_values.append(min(train_scaled_D[col]))
109     max_values.append(max(train_scaled_D[col]))
110
111 # SCRITTURA SU FILE DEI VALORI MIN&MAX
112 with open('min&max_values_dataset_out.txt', 'w') as f:
113     for item_min in min_values:
114         f.write(str(item_min) + " ")
115     f.write("\n")
116     for item_max in max_values:
117         f.write(str(item_max) + " ")
118
119 # FEATURE IMPORTANCE
120 feature_importance(train_scaled_D, training_set_labels)
121
122
123 # validation con k-fold cross-validation
124 n_folds = 10
125
126
127 # CLASSIFICATORE
128 classificatore = MLPClassifier()
129 #             ['tanh', 'relu']
130 pg = {'activation': ['relu'], 'learning_rate': ['invscaling', 'adaptive',
```

5. IL NOSTRO APPROCCIO

```
131 'constant'],
132     'solver': ['adam', 'lbfgs', 'sgd'],
133     'learning_rate_init': [0.01, 0.05, 0.1, 0.15, 0.2],
134     'hidden_layer_sizes': [3, 5, 10, 15, 17],
135     'max_iter': [2500, 5000, 10000]}
136 bestMLPparam = grid("marcosmiles_dataset.csv", classificatore,
137                     pg, n_folds, train_scaled_D,
138                     training_set_labels.values.ravel())
139 print("I migliori parametri per MLP sono:")
140 print(bestMLPparam)
141
142
143 classificatore = MLPClassifier(activation=bestMLPparam['activation'],
144                               learning_rate=bestMLPparam['learning_rate'],
145                               solver=bestMLPparam['solver'],
146                               learning_rate_init=bestMLPparam['learning_rate_init'],
147                               max_iter=bestMLPparam['max_iter'],
148                               hidden_layer_sizes=bestMLPparam['hidden_layer_sizes'])
149 classificatore.fit(train_scaled_D, training_set_labels.values.ravel())
150 y_pred = classificatore.predict(test_scaled_D)
151 precision, recall, fscore, support = score(test_set_labels, y_pred)
152 accuracy = accuracy_score(test_set_labels, y_pred)
153 print("MLP")
154 print("Accuracy")
155 print(accuracy)
156 print('precision: {}'.format(precision))
157 print('recall: {}'.format(recall))
158 print('fscore: {}'.format(fscore))
159 print("Weight e Bias")
160 # The ith element in the list represents the weight W matrix
161 #     corresponding to layer i.
162 print(classificatore.coefs_)
163 # The ith element in the list represents the bias B vector
164 #     corresponding to layer i + 1.
165 print(classificatore.intercepts_)
166
167 # SCRITTURA SU FILE DELLA PRECISION
```

```
168 with open('precision_out.txt', 'w') as aw:
169     aw.write("Accuracy: ")
170     aw.write(str(accuracy))
171     aw.write("\n")
172     aw.write("Precision: ")
173     aw.write(str(precision))
174     aw.write("\n")
175     aw.write("Recall: ")
176     aw.write(str(recall))
177     aw.write("\n")
178     aw.write("fscore: ")
179     aw.write(str(fscore))
180     aw.write("\n")
181
182 # SCRITTURA SU FILE DEI WEIGHTS
183 with open('weights_out.txt', 'w') as fw:
184     for w_layer in classificatore.coefs_:
185         fw.write(str(w_layer))
186         fw.write("\n")
187         # new line \n identifies the starting of a new layer
188
189 # SCRITTURA SU FILE DEI BIAS
190 with open('bias_out.txt', 'w') as fb:
191     for b_layer in classificatore.intercepts_:
192         fb.write(str(b_layer))
193         fb.write("\n")
194         # new line \n identifies the starting of a new layer,
195         # starting from the (i+1)th layer
196
197 # SCRITTURA SU FILE DEL TEMPO DI FINE APPRENDIMENTO
198 with open('learning_time.txt', 'a+') as lt:
199     now = datetime.now()
200     # dd/mm/YY H:M:S
201     dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
202     lt.write(str("Data Fine Learning: " + dt_string + "\n"))
```

5.5.4 Metodo di scaling

La tipologia di scaling utilizzata nello script del capitolo precedente è la **Normalizzazione MinMax** dato che, come già spiegato nel paragrafo dedicato allo scaling [5.4], è l'ideale per scalare i dati preparandoli ad operazioni di comparazione. Nel nostro caso infatti, saranno queste le operazioni eseguite con i dati istruiti. Inoltre, i valori minimi e massimi dei dati scalati sono salvati su file, così da permettere l'accesso a tali informazioni da programmi esterni allo script di addestramento assicurandosi di utilizzare i valori dei dati corretti, dato che l'addestramento è effettuato usando l'80% del dataset registrato. Il restante 20% è utilizzato esclusivamente per il testing.

5.5.5 Funzione di attivazione

Le funzioni di attivazione utilizzate inizialmente sono state le funzioni *tanh* e *ReLU*. Successivamente, avendo riscontrato maggiore successo con la funzione *ReLU* abbiamo deciso di supportare esclusivamente questa funzione.

5.5.6 Feature Recognition

Una volta completato l'addestramento della rete neurale va implementato l'utilizzo di essa. Essendo il sistema MarcoSmiles implementato in unity, e di conseguenza con il linguaggio C#, non è più necessario l'utilizzo del linguaggio Python.

```
1 using System.Collections.Generic;
2 using System.Globalization;
3 using System.IO;
4 using System.Linq;
5 using UnityEngine;
6
7 public static class TestML
8 {
9     private static List<List<float>> W1 = new List<List<float>>();
10
11     private static List<float> B1 = new List<float>();
```

5. IL NOSTRO APPROCCIO

```
12
13 private static List<List<float>> W2 = new List<List<float>>();
14
15 private static List<float> B2 = new List<float>();
16
17 public static System.DateTime DateLatestLearning;
18
19
20 /// <summary>
21 /// ReadArraysFromFormattedFile Restituisce una lista di array float.
22 /// Gli array sono inseriti nella lista nell'ordine in cui vengono
23 /// letti dal file. La lista e' formattata nel seguente modo:
24 /// Finche' leggendo il file vengono letti vettori, allora questi
25 /// vettori vengono aggiunti alla lista e restituiti.
26 /// Nel momento in cui la funzione capisce di aver letto una matrice,
27 /// inserisce nella lista un array vuoto [].
28 ///
29 /// Ad esempio:
30 /// - nel file bias.txt, sono contenuti array. Ogni array rappresenta un bias:
31 /// Il primo array rappresenta B1; il secondo array B2 etc...
32 ///
33 /// - nel file weights.txt, sono contenute matrici, ognuna contiene una
34 /// lista di array. Quindi tutti gli array contenuti nella lista restituita,
35 /// finche' non si legge un array vuoto [], faranno parte della prima
36 /// matrice letta. In questo modo, si formatta la lista in n blocchi di
37 /// array, dove n e' il numero di matrici contenuti nel file:
38 /// Tutti gli array contenuti nel primo blocco di array (quelli che si
39 /// trovano prima del primo array vuoto[]) rappresentano la matrice W1,
40 /// Tutti gli array contenuti nel secondo blocco di array (quelli che
41 /// si trovano dopo il primo array vuoto[] e prima del secondo array
42 /// vuoto []) rappresentano la matrice W2
43 /// etc...
44 /// </summary>
45 /// <param name="name">Nome del file da aprire</param>
46 /// <returns>
47 /// Una lista di float contenente i numeri letti da file. La lista e
48 /// formattata logicamente in modo tale da poter costruire gli
```

```
49  /// <eventuali array contenuti nel file.
50  /// </returns>
51  private static List< List<float> > ReadArraysFromFormattedFile(string name)
52  {
53      //stringa
54      var text = FileUtils.LoadFile(name);
55
56      if (text == null)
57          return null;
58
59      DateLatestLearning = File.GetLastWriteTime(FileUtils.GeneratePath(name));
60
61      text = text.Replace("[", "");
62      text = text.Replace("\n", "");
63
64      string[] vettori = text.Split(' ');
65
66      //lista degli array letti, i valori degli array sono stringhe
67      List< List <string> > temp = new List< List<string> >();
68
69      foreach (string vettore in vettori)
70      {
71          //divide per ' ' e mette l'array di stringhe all'interno della lista temp
72          temp.Add(vettore.Split(' '))
73              .Select(tag => tag.Trim())           //elimina le entrate vuote
74              .Where(tag => !string.IsNullOrEmpty(tag)).ToList();
75      }
76
77      //lista degli array letti che verra' restituita
78      List<List<float>> listOfReadArrays = new List< List<float> >();
79
80
81      //Converte tutti i valori degli array letti, da string a float.
82      foreach ( var arr in temp)
83      {
84          float[] t = new float[arr.Count];
85          for (int i = 0; i < arr.Count; i++)
```



```
86         {
87             t[i] = float.Parse(arr[i], CultureInfo.InvariantCulture);
88         }
89         listOfReadArrays.Add(t.ToList());
90     }
91     return listOfReadArrays;
92 }
93
94
95 /// <summary>
96 /// Riempie le Matrici W1 ; B1 ; W2; B2.
97 /// Usa il metodo ReadArraysFromFormattedFile, per leggere da un file
98 /// una lista di arrays di tipo float.
99 /// Questa lista restituita e' formattata logicamente.
100 /// </summary>
101 public static bool Populate()
102 {
103     B1.Clear(); B2.Clear(); W1.Clear(); W2.Clear();
104
105     List<List<float>> biasArrays =
106         ReadArraysFromFormattedFile("bias_out.txt");
107     if (biasArrays == null)
108         return false;
109
110     B1 = biasArrays.ElementAt(0);
111     B2 = biasArrays.ElementAt(1);
112
113     List<List<float>> weightsArrays =
114         ReadArraysFromFormattedFile("weights_out.txt");
115     if (weightsArrays == null)
116         return false;
117
118
119     //finche' non arrivo ad un array uguale a {} sono array che rappresentano W1
120     int j = 0;
121     while (Enumerable.SequenceEqual(weightsArrays.ElementAt(j),
122         new List<float> { } ) == false)
```

5. IL NOSTRO APPROCCIO

```
123     {
124         j++;
125     }
126
127     for(int i = 0; i < j ; i++)
128     {
129         W1.Add(weightsArrays.ElementAt(i));
130     }
131
132     // da j+1 alla fine sono array che rappresentano W2
133     int k = j+1;
134     while (Enumerable.SequenceEqual(weightsArrays.ElementAt(k),
135         new List<float> { } ) == false)
136     {
137         k++;
138     }
139
140     for (int i = 0 ; i < k - (j + 1) ; i++)
141     {
142         W2.Add(weightsArrays.ElementAt(j + 1 + i));
143     }
144
145
146
147     return true;
148
149 }
150
151 /// <summary>
152 /// Predice la nota associata alle features passato come parametro.
153 /// Usa le matrici W1, B1, W2 e B2 per effettuare i calcoli.
154 /// Restituisce un indice che va da 0 a 23, corrispondente alla nota da
155 /// suonare.
156 /// La funzione supporta il caso in cui il dataset contenga meno note
157 /// rispetto agli indici 0-23. Questo e' fatto calcolando in fase di
158 /// computazione la grandezza delle matrici W1, B1, W2, B2.
159 /// Conseguentemente, la funzione ReteNeurale, restituira' indici che
```

```
160     /// si trovano fra il range di note allenate.
161     /// </summary>
162     /// <param name="features">nota da trovare</param>
163     /// <returns>Id nota trovata</returns>
164     public static int ReteNeurale(float[] features)
165     {
166
167
168         float[] scaledFeatures = ScaleValues(features);
169         for (int i = 0; i < scaledFeatures.Length; i++)
170         {
171             features[i] = scaledFeatures[i];
172         }
173
174         /// output_hidden1 ha lo stesso numero di elementi di B1
175         var output_hidden1 = new float[B1.Count];
176         /// output_hidden2 ha lo stesso numero di elementi di B2
177         var output_hidden2 = new float[B2.Count];
178
179
180         float x, w, r;
181
182         for (int i = 0; i < output_hidden1.Length; i++)
183         {
184             output_hidden1[i] += B1.ElementAt(i);
185             for (int j = 0; j < features.Length; j++)
186             {
187                 x = features[j];
188                 w = W1[j][i];
189                 r = x * w;
190
191                 output_hidden1[i] += r;
192             }
193
194             if (output_hidden1[i] <= 0)
195                 output_hidden1[i] = 0;
196         }
```

5. IL NOSTRO APPROCCIO

```
197
198     for (int i = 0; i < output_hidden2.Length; i++)
199     {
200         output_hidden2[i] += B2.ElementAt(i);
201         for (int j = 0; j < output_hidden1.Length; j++)
202         {
203             x = output_hidden1[j];
204             w = W2[j][i];
205             r = x * w;
206
207             output_hidden2[i] += r;
208         }
209     }
210
211     float sum = 0;
212     foreach (var item in output_hidden2)
213     {
214         sum += Mathf.Exp((float)item);
215     }
216
217     var toRet = new float[output_hidden2.Length];
218     for (int i = 0; i < output_hidden2.Length; i++)
219     {
220         toRet[i] = Mathf.Exp((float)output_hidden2[i]) / sum;
221     }
222
223     return toRet.ToList().IndexOf(toRet.Max());
224 }
225
226
227
228
229     /// <summary>
230     /// Scala i valori convertendo le features non scalate in valori tra 0 e 1
231     /// </summary>
232     /// <param name="unscaledFeatures">features da scalare</param>
233     /// <returns>features scalate con valori tra 0 e 1</returns>
```

5. IL NOSTRO APPROCCIO

```
234 private static float[] ScaleValues(float[] unscaledFeatures)
235 {
236     var scaledFeatures = new float[unscaledFeatures.Length];
237     var minValues = new float[unscaledFeatures.Length];
238     var maxValues = new float[unscaledFeatures.Length];
239
240     string[] readText = File.ReadAllLines(FileUtils.
241         GeneratePath("min&max_values_dataset_out.txt"));
242
243     //primo elemento contiene riga contenente valori min
244     //secondo elemento contiene riga contenente valori max
245     string[] min = readText[0].Split(' ');
246     string[] max = readText[1].Split(' ');
247
248     for (int i = 0; i < unscaledFeatures.Length; i++)
249     {
250         minValues[i] = float.Parse(min[i], CultureInfo.InvariantCulture);
251         maxValues[i] = float.Parse(max[i], CultureInfo.InvariantCulture);
252     }
253
254     for (int i = 0; i < unscaledFeatures.Length; i++)
255     {
256         scaledFeatures[i] = (unscaledFeatures[i] - minValues[i]) /
257             (maxValues[i] - minValues[i]);
258     }
259     return scaledFeatures;
260 }
261 }
```

CAPITOLO 6

RISULTATI E DISCUSSIONI

In questo capitolo saranno discussi i risultati ottenuti durante gli studi del progetto, discutendoli. Nonostante i risultati particolarmente soddisfacenti che sono stati ottenuti, si sono presentati alcuni problemi, i quali saranno discussi. Tali problemi hanno portato quindi a diverse osservazioni per apportare correzioni o miglioramenti per quanto riguarda l'implementazione.

Gli eventi principali che abbiamo identificato e studiato sono tre:

- **Leggero Overfitting;**
- **Differenze tra dati scalati e non scalati;**
- **Differenze di tempo nella fase di learning;**

6.1 Analisi Overfitting e Differenze tra dati non scalati e dati scalati

In questa sezione analizzeremo grafici e dati riguardanti le accuracy e le **Confusion Matrix** di alcuni dei diversi learning effettuati.

6.1.1 1 Ottava con dati non scalati

Questa tabella raffigura i risultati del testing in fase di training.

Accuracy: 1.0				
Feature n.	Precision	Recall	fscore	
0	1.0	1.0	1.0	
1	1.0	1.0	1.0	
2	1.0	1.0	1.0	
3	1.0	1.0	1.0	
4	1.0	1.0	1.0	
5	1.0	1.0	1.0	
6	1.0	1.0	1.0	
7	1.0	1.0	1.0	
8	1.0	1.0	1.0	
9	1.0	1.0	1.0	
10	1.0	1.0	1.0	
11	1.0	1.0	1.0	

Figura 6.1: Accuracy di 1 ottava non scalata

Nel seguente grafico si può visualizzare l'effettiva accuratezza totale dei test in fase di training mostrata nell'immagine precedente. In queste matrici, le caselle alla *i-esima riga* ed *i-esima colonna* raffigurano la **predizione attesa**. Essendo tutti i dataset composti da 500 elementi per ogni nota, ed essendo il training suddiviso in 80% del dataset destinato al training ed il 20% restante destinato al testing, si avranno test con circa 100 elementi per nota. Nel nostro caso la matrice di confusione è una stima basata sui risultati dell'accuracy. La matrice di confusione ottenuta in fase di test è invece composta da valori reali delle stime ottenute. Si può notare come i risultati dell'accuratezza in fase di training sono sempre migliori rispetto ai risultati di accuratezza in fase di test. Questo problema è chiamato **Overfitting**. In fase di test, l'accuracy ottenuta è del **91.75%**, il quale è comunque un risultato molto alto, però il problema principale di questa rete neurale è che l'errore non è distribuito su tutte le note, ma si concentra su 2, la *nona* e la *decima*.

6. RISULTATI E DISCUSSIONI

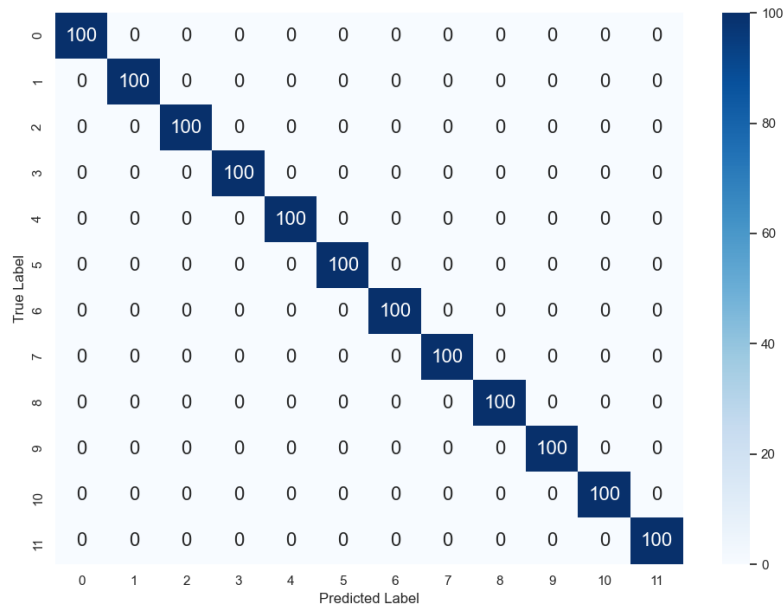


Figura 6.2: Confusion Matrix di dataset da 1 ottava non scalata, training

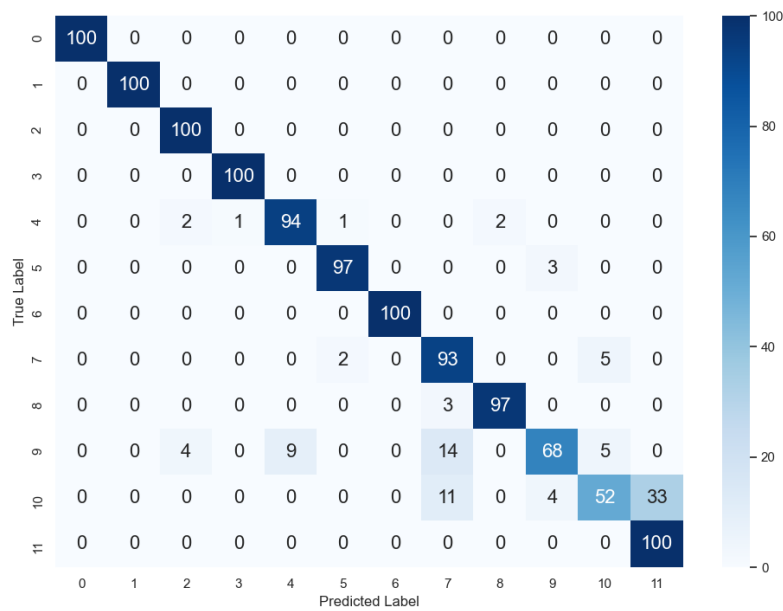


Figura 6.3: Confusion Matrix di dataset da 1 ottava non scalata, test

6.1.2 1 Ottava con dati scalati

Dati i risultati ottimi ma comunque non totalmente soddisfacenti (nella fase di test), abbiamo deciso di far uso del metodo di *scaling* della **normalizzazione MinMax** sullo stesso dataset del caso di studio precedente. Questo dovrebbe permetterci di ridurre l'errore, incrementando la percentuale di *accuracy* sia nel test della fase di training che nella fase di test.

Proprio come per il caso precedente l'*accuracy* del test in fase di training risulta del **100%**, valore che però non si ripete nella fase di test, dove l'*accuracy* ottenuta è del **94%**. Il guadagno, seppur minimo, ci consentirà un maggior successo nella predizione delle note, in particolare della *decima* nota, dove è stato riscontrato un guadagno del **20%** rispetto alla situazione precedente.

Accuracy: 1.0			
Feature n.	Precision	Recall	fscore
0	1.0	1.0	1.0
1	1.0	1.0	1.0
2	1.0	1.0	1.0
3	1.0	1.0	1.0
4	1.0	1.0	1.0
5	1.0	1.0	1.0
6	1.0	1.0	1.0
7	1.0	1.0	1.0
8	1.0	1.0	1.0
9	1.0	1.0	1.0
10	1.0	1.0	1.0
11	1.0	1.0	1.0

Figura 6.4: Accuracy di 1 ottava scalata

6. RISULTATI E DISCUSSIONI

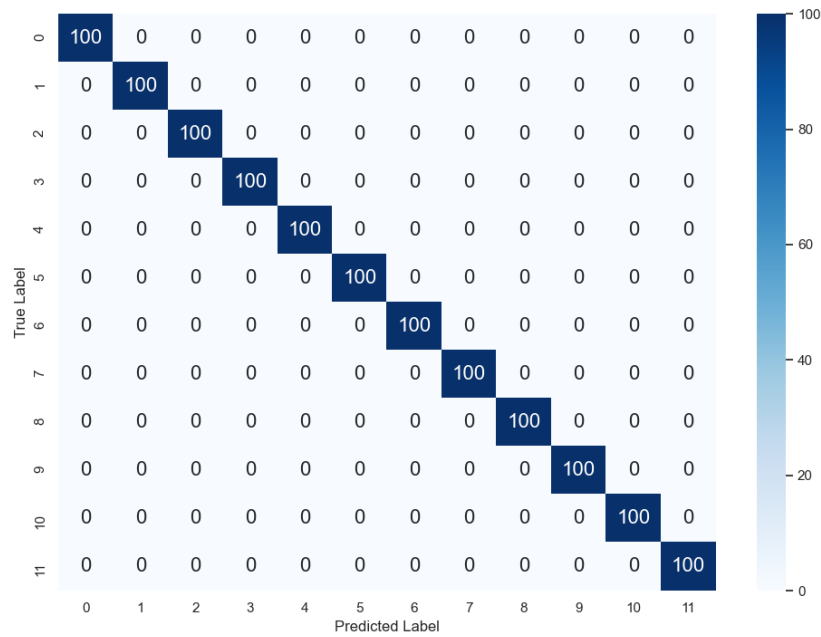


Figura 6.5: Confusion Matrix di dataset da 1 ottava scalata, training

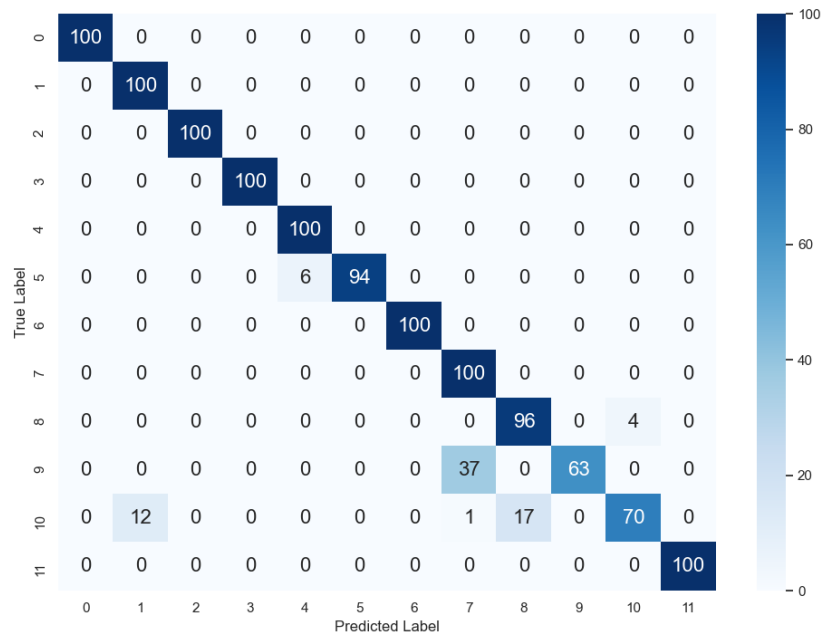


Figura 6.6: Confusion Matrix di dataset da 1 ottava scalata, test

6.1.3 2 Ottave con dati non scalati

Per un'esperienza migliore e meno limitata, il sistema *MarcoSmiles* offre la possibilità di effettuare addestramenti su dataset composti da 2 ottave, permettendo quindi di avere a disposizione 24 note da poter suonare. Questo vantaggio comporta però un abbassamento delle prestazioni generali del sistema, sia per funzioni inerenti al **Modulo Unity** (come ad esempio l'eliminazione di note dal dataset), che per quanto riguarda l'**addestramento** della rete. Bisogna considerare che, quando si addestrava una rete su un dataset composto da una sola ottava, l'addestramento veniva eseguito su file contenenti minimo 6000 classi di informazioni. Con dataset da due ottave, invece, la mole *minima* di dati **raddoppia**, arrivando ad un minimo di 12000 classi di informazioni.

Di seguito sono mostrati dati e **Confusion Matrix** ottenuti dal training di dataset contenenti **2 ottave non scalate**.

Accuracy: 1.0				
Feature n.	Precision	Recall	fscore	
0	1.0	1.0	1.0	
1	1.0	1.0	1.0	
2	1.0	1.0	1.0	
3	1.0	1.0	1.0	
4	1.0	1.0	1.0	
5	1.0	1.0	1.0	
6	1.0	1.0	1.0	
7	1.0	1.0	1.0	
8	1.0	1.0	1.0	
9	1.0	1.0	1.0	
10	1.0	1.0	1.0	
11	1.0	1.0	1.0	
12	1.0	1.0	1.0	
13	1.0	1.0	1.0	
14	1.0	1.0	1.0	
15	1.0	1.0	1.0	
16	1.0	1.0	1.0	
17	1.0	1.0	1.0	
18	1.0	1.0	1.0	
19	1.0	1.0	1.0	
20	1.0	1.0	1.0	
21	1.0	1.0	1.0	
22	1.0	1.0	1.0	
23	1.0	1.0	1.0	

Figura 6.7: Accuracy di 2 ottave non scalate

6. RISULTATI E DISCUSSIONI

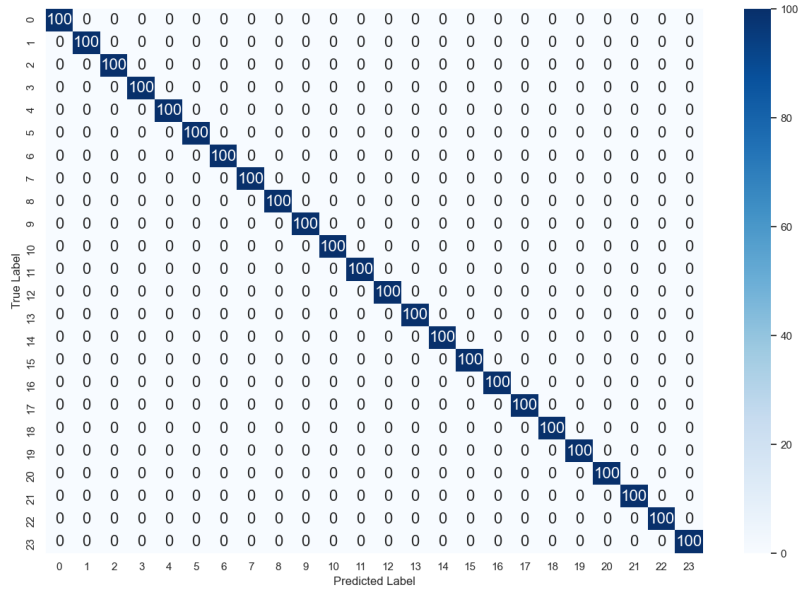


Figura 6.8: Confusion Matrix di dataset da 2 ottave non scalate, training

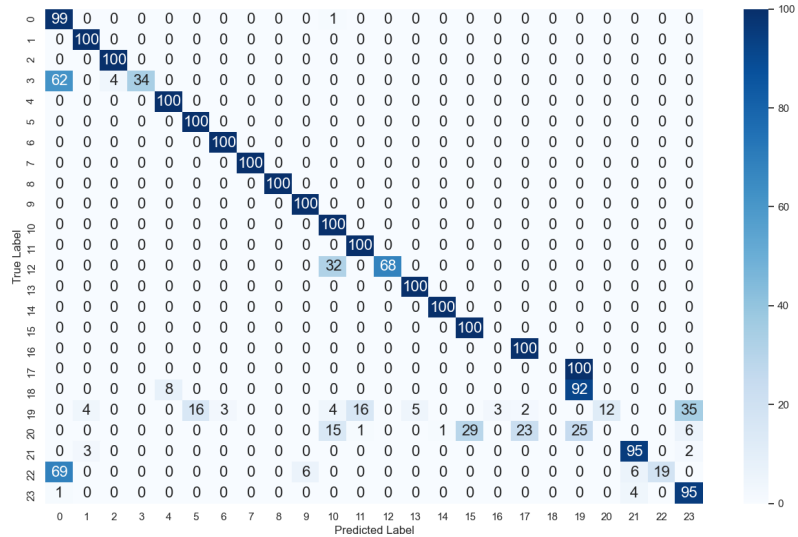


Figura 6.9: Confusion Matrix di dataset da 2 ottave non scalate, test

Esattamente come ci aspettavamo, i risultati di accuracy del test sono decisamente inferiori rispetto alle *accuracy* dei test su una sola ottava. Nonostante l'accuracy del **100%** nel test della fase di training, l'*accuracy*

6. RISULTATI E DISCUSSIONI

ottenuta nella fase di test è diminuita fino al **71.3%**. Il problema principale però è il totale erroneo riconoscimento di determinate posizioni, in questo caso di **5 note** della seconda ottava, ed uno scarso punteggio per una nota della prima ottava ed una nota della seconda ottava.

6.1.4 2 Ottave con dati scalati

Proprio come per i test su una sola ottava, abbiamo deciso di utilizzare lo scaling per **MinMax** anche per il training su due ottave, per valutare la validità dell'inclusione dello scaling nel prodotto finale.

Nonostante l'*accuracy* del test nel training con dati scalati sia impercettibilmente inferiore rispetto ad i test con dati non scalati, i risultati della fase di test dimostrano il contrario, con un incremento dell'*accuracy*. Dal **71.3%** iniziale, infatti, abbiamo ottenuto un'*accuracy* dell'**80%**. Il risultato principale però è il riconoscimento parziale di alcune note che prima avevano una probabilità di essere riconosciute quasi nulla. Inoltre, a differenza del caso di studio precedente, le note totalmente ignorate, nonostante il riconoscimento minimo, sono diventate solo **tre** anziché **cinque**.

Accuracy: 0.9995833333333334			
Feature n.	Precision	Recall	fscore
0	1.0	1.0	1.0
1	1.0	1.0	1.0
2	1.0	1.0	1.0
3	1.0	1.0	1.0
4	1.0	1.0	1.0
5	1.0	1.0	1.0
6	1.0	1.0	1.0
7	1.0	1.0	1.0
8	1.0	1.0	1.0
9	1.0	1.0	1.0
10	1.0	1.0	1.0
11	1.0	1.0	1.0
12	1.0	1.0	1.0
13	1.0	1.0	1.0
14	1.0	1.0	1.0
15	1.0	1.0	1.0
16	1.0	1.0	1.0
17	1.0	1.0	1.0
18	0.99009901	1.0	0.99502488
19	1.0	0.99	0.99497487
20	1.0	1.0	1.0
21	1.0	1.0	1.0
22	1.0	1.0	1.0
23	1.0	1.0	1.0

Figura 6.10: Accuracy di 2 ottave scalate

6. RISULTATI E DISCUSSIONI

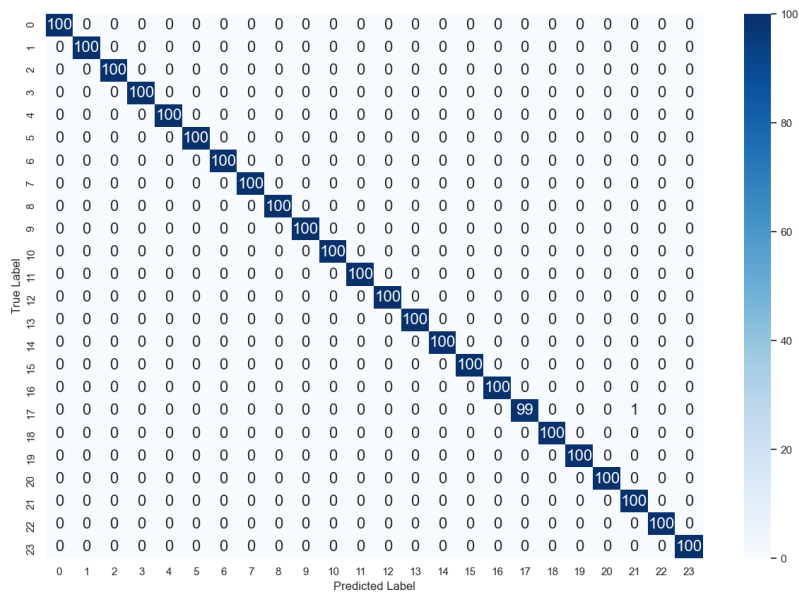


Figura 6.11: Confusion Matrix di dataset da 2 ottave scalate, training

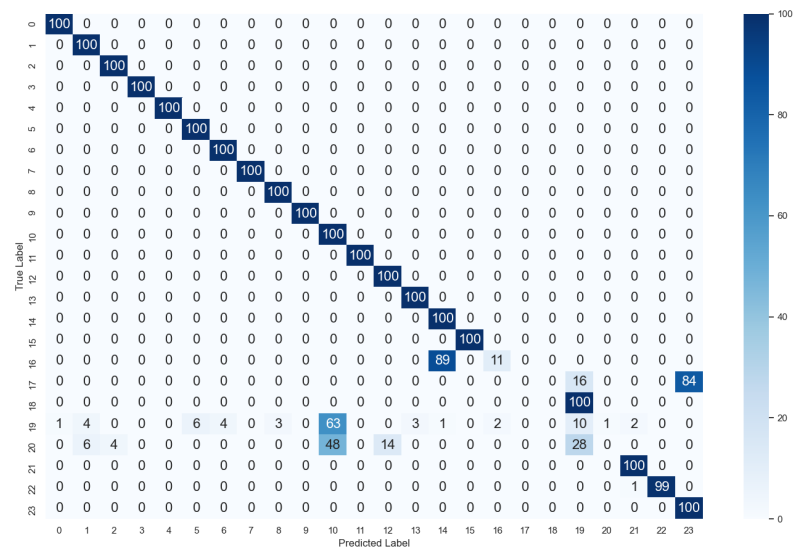


Figura 6.12: Confusion Matrix di dataset da 2 ottave scalate, test

6.2 Differenze di tempo nella fase di learning

Il training di una rete neurale è un procedimento complesso, che richiede una modesta quantità di tempo. Questa quantità di tempo dipende direttamente particolarmente dalla mole di dati che l'algoritmo di training andrà ad utilizzare: maggiori è tale mole, e maggiore sarà il tempo necessario al completamento dell'apprendimento. La metodologia di programmazione parallela ci aiuta particolarmente in situazioni di questo tipo, riducendo drasticamente i tempi necessari.

Come mostreremo in questa sezione, l'utilizzo di tale metodologia permette di accorciare la fase di training, consentendoci di utilizzare la rete neurale il prima possibile.

Nella seguente tabella sono mostrati diversi dataset da noi addestrati, con le relative dimensioni, i tempi di training ed il numero di core utilizzati.

Nome Dataset	Range Note	Righe per Nota	Durata Training (minuti)	Numero Jobs (cores)	Accuracy Training	Scaling Y/N
Dataset_20Oct_1	0-23	500	140	20	0,9975	N
Dataset_20Oct_2	0-23	500	140	20	1	N
Dataset_20Oct_3	0-23	500	130	20	1	N
Dataset_20Oct_3_Scaled	0-23	500	70	20	0,9998	Y
Dataset_10Oct_1	0-11	500	20	20	1	N
Dataset_10Oct_1_Scaled	0-11	500	19	20	1	N
Dataset_10Oct_2	0-11	500	50	6	0,9985	Y
Dataset_20rows_1	0-11	20	15	5	0,7	N
Dataset_20rows_2	0-11	20	10	5	0,97	N
Dataset_20rows_3	0-11	20	7	5	1	Y
Dataset_20rows_4	0-23	20	10	5	1	Y
Dataset_20rows_5	0-23	20	14	5	1	N

Figura 6.13: Tabella datasets

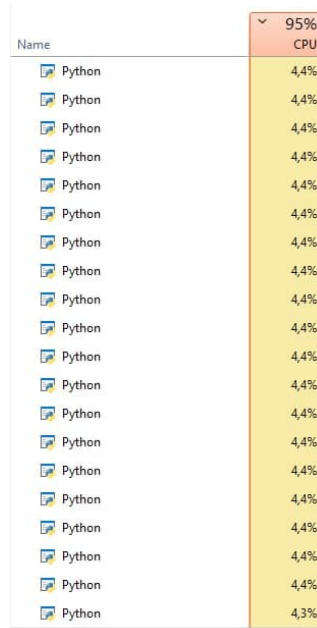
Gran parte dei dataset sono stati addestrati provando due funzioni di attivazione: **ReLU** e **TanH**. Successivamente, nella versione finale dello script di learning, l'unica funzione di attivazione usata è **ReLU**. Questa decisione è stata presa per ridurre il tempo di training, e dato che era la funzione di attivazione che ci ha restituito risultati più soddisfacenti. ”*Dataset_20Oct_3_Scaled*” è il dataset addestrato con l'ultima versione dello

6. RISULTATI E DISCUSSIONI

script di training, difatti si può notare come, con lo stesso numero di core della sua versione non scalata, abbia impiegato circa la **metà** del tempo (70 minuti anziché 130). Questo perché vengono effettuate la metà delle operazioni. Lo stesso principio vale per le dimensioni del dataset, i dataset da 1 sola ottava impiegano notevolmente meno rispetto ai dataset che supportano 2 ottave.

Differenze sostanziali di tempo le possiamo notare con il numero differente di core. Dataset addestrati con 20 core, infatti, impiegano meno di un terzo del tempo rispetto ai dataset addestrati con 6 core. Altri test non riportati nella tabella mostrano che il tempo impiegato per completare l'addestramento è inversamente proporzionale al numero di core utilizzati. Se infatti andassimo ad addestrare un dataset con 5 core, l'addestramento di tale impiegherebbe 4 volte il tempo di un addestramento effettuato sullo stesso dataset, ma con 20 core. Bisogna quindi considerare che per dataset come "*Dataset_2Oct_3_Scaled*", anziché soli 70 minuti, ne saranno necessari circa 280, ovvero circa 4 ore e mezza. Questo considerando una sola registrazione per nota.

Nei test da noi effettuati con l'utilizzo di 20 core, la quantità reale di core della cpu era 24, ma è stato necessario trovare un limite massimo di core utilizzabili, per evitare l'impiego totale della cpu, che comporterebbe paradossalmente una perdita di prestazioni, allungando i tempi di training. Vengono quindi utilizzati solo **5/6** dei core disponibili.



Name	95% CPU
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,4%
Python	4,3%

Figura 6.14: Utilizzo della cpu durante training con 20 core

Come si può notare nella figura precedente, durante il training l'utilizzo della cpu è pari al **95%**, percentuale che seppure permetta l'utilizzo della macchina fluido per azioni standard, limita l'utilizzo di esso per compiti più onerosi. È quindi da tenere a mente che durante la fase di training *non sarà possibile* utilizzare la propria macchina al pieno delle sue potenzialità.

6.3 CONCLUSIONI

Nonostante i problemi riportati nei capitoli precedenti, però, la causa di errori principale per sistemi come questo è proprio il fattore umano. Per quanto precisa una rete neurale possa essere, ci sarà sempre la possibilità di un errore di riconoscimento causato dall'utente stesso: una transizione tra una posizione ed un'altra che combacia con una configurazione registrata, l'assunzione di una posizione troppo differente da quelle utilizzate in fase di training, o addirittura la registrazione di un dataset in maniera errata. Questi sono aspetti di cui bisogna tener conto quando si interagisce con una rete neurale addestrata per il gesture recognition.

Tralasciando questi scenari, l'utilità di un tool del genere non è poca; si pensi

ad un *insegnante di chitarra*, che durante la sua lezione spiega gli accordi al suo studente. Come per MarcoSmiles, gli accordi possono essere visti come **configurazioni**, proprio come quelle spiegate nel capitolo 5.1. Uno studente alle prime armi non avrà molta dimestichezza con questi nuovi gesti che andrà ad imparare, ma col passare del tempo, e con l'esercizio, riuscirà a padroneggiarli. Uno scenario simile è del tutto realizzabile con Marcosmiles a patto di differenze sostanziali nella conformazione dei due utenti. Abbiamo infatti notato come, con una rete neurale addestrata sul modello delle mani di un individuo, potesse essere utilizzata a pieno da un secondo individuo, senza riscontrare problemi. Si immagini quindi un insegnante che registra il proprio dataset ed esegue l'addestramento, per poi farlo utilizzare ad i proprio studenti: la somiglianza con l'insegnante di chitarra è tale che, eliminate le differenze tra i due strumenti e le configurazioni utilizzate, l'esperienza è *identica*.

6.4 Lavori futuri

In questa sezione saranno proposti alcuni lavori futuri che potrebbero apportare ulteriori miglioramenti al sistema MarcoSmiles, migliorando l'esperienza complessiva.

- **Utilizzo di più sensori:** L'implementazione del supporto di più sensori (dello stesso tipo o differenti), anche con uso in contemporanea, permetterebbe la cattura un maggior numero di informazioni sulle gestures dell'utente, al fine di migliorare sia le predizioni della rete neurale, che la modalità di interazione con il sistema stesso.
- **Utilizzo di altre features:** Si potrebbe pensare di aggiungere altre informazioni sulla configurazione delle mani. Quest'opzione è valida sia nel caso si disponga di un singolo sensore, sia nello scenario in cui il sistema dispone di più sensori da utilizzare. Ciò renderebbe il sistema altamente personalizzabile, aumentando di fatto il numero di gestures che l'utente potrebbe utilizzare.

- ***Aumentare il volume del dataset:*** È stato evidenziato come con l'aumentare del volume del dataset sia possibile ridurre il problema dell'*overfitting*, portando quindi un incremento della *accuracy* reale.
- ***Estensione del range di note disponibili:*** Si potrebbe estendere il range di note oltre le 2 ottave. Questo porterebbe ad un'esperienza più prossima ad uno strumento reale.
- ***Addestramento delle reti neurali tramite servizi cloud:*** Nonostante la macchina principale utilizzata per gli studi trattati in questa tesi sia una macchina di fascia alta, è pur sempre limitata ad un ambiente domestico. Inoltre bisogna tener conto che i computer medi non hanno caratteristiche corrispondenti alla suddetta macchina, adeguate ad eguagliare tali prestazioni. Per minimizzare i tempi di training si potrebbe implementare l'utilizzo di servizi cloud che mettono a disposizione l'enorme potenza computazionale delle loro infrastrutture, come ad esempio Google Colab [31]. Questo permetterebbe di ridurre drasticamente i tempi di training.

BIBLIOGRAFIA

- [1] Maxine Sherrin, August de los Reyes: Predicting the past Web Directions, 2008
- [2] Ditte Hvas Mortensen, Natural User Interfaces – What are they and how do you design user interfaces that feel natural Interaction Design Foundation, 2020,
- [3] Computer Music - https://en.wikipedia.org/wiki/Computer_music
- [4] Pietro Grossi Computer Music - <http://www.hackerart.org/corsi/aba02/taddepera/computer-music.htm>
- [5] Natural User Interfaces to Support and Enhance Real-Time Music Performance - Rocco Zaccagnino, Roberto De Prisco, Delfina Malandrino, Gianluca Zaccagnino
- [6] Ariana Grande Sings Harmonies with Mimu Gloves, San Jose April 12th, 2015 - <https://www.youtube.com/watch?v=sa6h6wZpE30>
- [7] Mi.Mu Gloves - <https://mimugloves.com/>
- [8] Accessible Digital Musical Instruments — A Review of Musical Interfaces in Inclusive Music Practice, Emma Frid KTH Royal Institute of Technology, Lindstedtsvägen 3, 100 44 Stockholm, Sweden

BIBLIOGRAFIA

- [9] Theremin - <https://it.wikipedia.org/wiki/Theremin>
- [10] Mellotron - <https://it.wikipedia.org/wiki/Mellotron>
- [11] Omnichord - <https://it.wikipedia.org/wiki/Omnichord>
- [12] AUMI Software - <http://aumiapp.com/about.php>
- [13] KAiKu Music Gloves - <https://www.kaikumusicglove.com/>
- [14] Microsoft Kinect https://it.wikipedia.org/wiki/Microsoft_Kinect
- [15] Real-Time Musical Conducting Gesture Recognition Based on a Dynamic Time Warping Classifier Using a Single-Depth Camera - Fahn Chin-Shyurng, Shih-En Lee and Meng-Luen Wu, Department of Computer Science and Information Engineering, National Taiwan University of Science and Technology, Taipei 10607, Taiwan Dalian University of Technology
- [16] AeroMidi - <https://aeromidi.net/index.php>
- [17] Hand Gesture Recognition with Leap Motion - Youchen Du, Shenglan Liu, Lin Feng, Menghui Chen, Jie Wu
- [18] Virtual Piano - <https://gallery.leapmotion.com/virtual-piano-for-beginners/>
- [19] Natural User Interface - Next Mainstream Product User Interface, WeiyuanLiu, Guilin University Of Electronic Technology, School Of Art&Design Industiral Design, Guilin China
- [20] Leap Motion Controller by UltraLeap, https://www.ultraleap.com/datasheets/Leap_Motion_Controller_Datasheet.pdf
- [21] Wikipedia Feature Scaling https://en.wikipedia.org/wiki/Feature_scaling
- [22] Script lifecycle overview - <https://docs.unity3d.com/Manual/ExecutionOrder.html>
- [23] kinect figure <https://creativevreality.wordpress.com/2016/01/26/setting-up-the-kinect-on-osx-el-capitan/>
- [24] wiimote figure https://en.wikipedia.org/wiki/Wii_Remote/

BIBLIOGRAFIA

- [25] tanhrelusigmoid figure <https://link.springer.com/article/10.1007/s11042-020-08740-w/figures/7>
- [26] <https://medium.com/@PABTennnis/how-to-create-a-neural-network-from-scratch-in-python-math-code-fd874168e955>
- [27] Foto margine di errore Frequenze di campionamento <https://www.headphonesty.com/2019/12/dacs-guide-part-1/>
- [28] Unity ML-Agents, <https://unity.com/products/machine-learning-agents>
- [29] Microsoft ML.Net, <https://dotnet.microsoft.com/apps/machinelearning-ai/ml-dotnet>
- [30] Python Scikit-Learn, <https://scikit-learn.org/stable/>
- [31] Google Colab, <https://colab.research.google.com/notebooks/intro.ipynb>
- [32] Scaling, <https://curiously.com/posts/hackers-guide-to-data-preparation-for-machine-learning/>
- [33] Tree Figure, https://www.researchgate.net/figure/Basic-structure-of-a-decision-tree-All-decision-trees-are-built-through-recursion_fig3_295860754
- [34] Random Forest https://www.researchgate.net/publication/265254799_Aggregated_Recommendation_through_Random_Forests